REMORA: Governed Execution Assurance for Tool-Using AI Agents

Stian Skogbrott*
Luftfiber AS
https://github.com/darklordVirtual/REMORA-research

Preprint v0.12.0, revision 2026-10-19; repository release tag v0.12.0
compiled October 19, 2026 by CI from the repository LaTeX source

Abstract

REMORA is a governed execution-assurance layer for tool-using AI agents. An agent's intention is not an effect. A deterministic policy gate routes each proposed call to ACCEPT, VERIFY, ABSTAIN or ESCALATE before dispatch. A signed ToolSpec states what an approval covers, and an exact-call `ExecutionLease` binds that approval to one tool, argument set, target and policy bundle. A single-use grant is consumed at an enforcement point that re-evaluates policy. The lifecycle records *authorized*, *dispatched*, *executed* and *verified effect* as separate claims. Model consensus cannot override a hard block. The thesis is architectural: authority to act comes from signed, deployment-owned artifacts and a deterministic gate, never from the model's own judgment, and each step of that authority can be checked afterwards.

Its mechanisms are individually known: capabilities with caveats, reference monitors, policy engines and tamper-evident logs. The contribution is a systems artifact that integrates them, together with an evaluation method of sealed, pre-registered tracks whose misses and negative results are published.

On a leakage-controlled 700-task tool-call simulator benchmark (effective $N = 70$) the full gate produced zero unsafe acceptances, 0 of 56 harmful templates (Wilson 95% upper bound 6.4% over those templates). Keyword baselines reached 1.4%, a difference that is not significant, and a component ablation attributes the result to no single component (CLAIM-001, CLAIM-020). On 208 AgentHarm scenarios [5] an imported intent-gating run made no false authorization but also blocked every benign counterpart, so it shows no discrimination (CLAIM-002). With every state verdict UNKNOWN, bounded verification let 60/60 valid-identifier reads proceed autonomously, against 0/60 without validators, a baseline that is zero by construction (CLAIM-014).

Sealed BFCL v4 tracks measure routing within one tool catalogue. REMORA routes annotated calls; it does not generate them, and each wrong call is another task's correct call substituted into the task, almost always a tool the task never offered. In a sealed C-ext3 track, declared tool contracts and task intents re-derived from the task text accepted 0/500 wrong calls (Wilson 95% upper bound 0.76% by row, 3.5% clustered by tool). Four of the seven pre-registered targets were missed. One of them is a safety target: 2 of 199 constructed wrong-tool calls were accepted (1.005%, against a bar of at most 1%). Legitimate read autonomy was 26.6% against a 75% target, and both argument-routing targets were missed, so the pre-registered joint hypothesis was not supported (CLAIM-019). A final sealed track, C-ext4, spent the remaining BFCL v4 reserve (1,562 episodes) on a revised bundle. It accepted 29.9% of correct calls against 6.9% for the C-ext3 bundle and accepted no wrong call, as did a refuse-all control. The gain sits on tools seen during development, so transfer to new tool families is untested (CLAIM-021). The earlier sealed track's 10.9% wrong-call rate, measured on another sample without contracts, is kept on record and is not a baseline for C-ext3 (CLAIM-018, superseded by CLAIM-019).

Each result cites its claim-register entry, and machine-readable capability and boundary declarations state what the system establishes and what it does not. A post-hoc attack evaluation of the execution chain, on the author's own catalogue, is in §15. REMORA ran none of 43 chain attacks and a policy-only gateway ran 33, while the secret reached the sink in all 11 exfiltrations built from allowed calls, on both. With an opt-in flow check in the decision and context labels the harness assigns, REMORA ran none of 10 exfiltrations and 2 of 4 benign con-

*Developer and maintainer of REMORA. Luftfiber AS. Contact: support@luftfiber.no.

trols. One frozen interoperability contract has two external fixture-level runs that meet the repository's independence rule; production enforcement remains unestablished, and the declared deployment status is `SHADOW_ONLY`: shadow-mode operation is permitted, enforcement is not.

Keywords: agentic AI safety, AI control, agent security, governance overlay, policy-as-code, selective prediction, uncertainty routing, audit governance, human-in-the-loop.

Identifiers used in this paper. The repository's registers and plans use short identifiers, which this paper keeps so that every statement can be traced.

CLAIM-NNN an entry in the claim register, `docs/assurance/claim_register_v1.yaml`. Each entry binds a stated result to its artifact, sample size, caveat and reproduce command.

REM-NNN an item in the remediation register, `docs/assurance/remediation_register.yaml`. REM-021, for example, is the independent human review that is still open.

CAP-NNN an entry in the capability register, `docs/assurance/capability_register_v1.yaml`.

RMR-CR-NNN a finding of the producer-owned security review of 2026-10-07 (`docs/assurance/reviews/2026-10-07-272b6c56/`).

NEGATIVE_RESULTS.md §N a numbered entry in the repository's record of negative results, falsified hypotheses and limitations.

SAP vN version N of the pre-registered statistical analysis plan (`docs/assurance/statistical_analysis_plan*.md`). SAP v5 governs C-ext3, SAP v6 governs C-ext4, and SAP v7 is a draft.

C-ext2, C-ext3, C-ext4 three sealed evaluation tracks on disjoint rows of BFCL v4, each run once. C-ext2 measured the routing pipeline (CLAIM-018, superseded by CLAIM-019 and kept on record; its sample differs, so it is not a baseline for C-ext3). C-ext3 added declared tool contracts and task intents (CLAIM-019). C-ext4 spent the remaining reserve on a revised bundle (CLAIM-021).

Arm F, F1, V2D evaluation arms. Arm F is the confirmatory C-ext3 configuration, with contracts, intents and the semantic-authority floor. In C-ext4, F1 is that same frozen bundle and V2D is the revised bundle v2 with deterministic derivation receipts, the confirmatory arm.

E-ECB, E-FA, E-EE, E7 frozen interoperability contracts for exact-call binding, fresh authority, effect evidence and the runtime capability surface (§4.3). CP-F1 to CP-F4 are the four findings of the contract probes against the federation transport.

Status values `NOT_ESTABLISHED`, `NOT_INDEPENDENT`, `EXTERNALLY_VERIFIED` and `INDEPENDENT` are status values in the capability, boundary and interoperability declarations (§4.3, §4.4).

SHADOW_ONLY the declared deployment status. Shadow-mode operation is permitted and enforcement is not, until the remaining release gates close (`docs/assurance/release_gates.md`).

AROMER, AII AROMER is the experimental learning layer; AII, its Autonomous Intelligence Index, is a researcher-defined composite score whose phase labels (WARMUP, LEARNING, CAPABLE, TRAINED) are qualitative (§16).

1 Introduction

An operator asks a fleet assistant for the status of vehicle `V-0042`. The assistant proposes `get_vehicle(vehicle_id="V-0042")`: the right tool and an identifier drawn from the request. REMORA first routes the read to VERIFY because the identifier has not been confirmed against the system of record. A bounded lookup confirms it; the full policy gate runs again on the fresh observation and can return ACCEPT. The assistant gets a path to a useful answer without treating an unverified value as authority (§14).

The research hypothesis is that agent autonomy becomes more useful when uncertainty determines *what to check*, while deployment-owned authority determines *what may run*. These are different jobs. Deterministic rules bind a call to a declared tool, permitted target and policy, and a hard block cannot be voted away. Probabilistic disagreement and evidence signals can help route an ambiguous proposal toward verification or review; in REMORA they are studied on a separate research path, not entrusted with execution permission. This separation lets us measure routing quality without mistaking model confidence for authority.

The clearest test of that hypothesis concerns a call that passes structural checks but uses the wrong tool for the task. In a sealed C-ext3 study of 2,799 episodes, declared tool contracts and task intents re-derived from the task text reduced acceptance of substituted wrong calls to 0/500 (Wilson 95% upper bound 0.76% by row, 3.5% clustered by tool). The arm comparison recorded 24, then 6, then 0 accepts as semantic

authority was introduced; the paired test was run only after the fact (§11.6, CLAIM-019). The promising mechanism is explicit semantic authority, not a more confident model vote. Its measured property is contract-authorized read, not guaranteed task success or live deployment performance. Its current price matters: the deterministic intent extractor allowed only 26.6% of legitimate reads to proceed autonomously in that study, so the pre-registered joint hypothesis, which also required 75% read autonomy, was not supported. The same run accepted 2 of 199 constructed wrong-tool calls against a safety bar of at most 1% and missed both argument-routing targets, four of its seven targets in all (§11.6). A later sealed track raised correct-call acceptance from 6.9% to 29.9% with a revised bundle and no wrong call accepted, mostly on tools seen in development; a refuse-all control also accepted no wrong call (§11.6, CLAIM-021). Bounded verification offers another path to useful autonomy when task state is unknown (§14, CLAIM-014).

REMORA places a decision boundary between a proposed call and its effect. Deployment-owned tool definitions and policy determine what the call is allowed to do. The gate may ACCEPT it, require a bounded lookup (VERIFY), decline autonomous action (ABSTAIN), or send it to a person (ESCALATE). A model's confidence cannot override a hard policy block. ACCEPT permits the next authorization step; it says nothing about whether the action is correct. Execution still requires authority bound to that exact call, a single-use grant and another policy check at dispatch.

That distinction continues after the call runs. Authorization, dispatch, execution and verified effect are recorded separately; a successful transport response does not establish what changed in the target system. The decision record also states which parts of the trust base its evidence covers. This makes it possible to ask a sharper question than whether an agent *wanted* to act: what authorized the action, what ran, and what evidence supports the reported outcome?

A second design decision constrains the first: the boundary itself is declared, not implied. Machine-readable capability and boundary declarations (§4.4) state which properties REMORA establishes at the governed boundary and under which execution profile. Explicit claim ceilings record what remains unestablished: alternate credential paths, and the privileged effects an authorized tool's implementation can reach. REMORA therefore does not claim that all agent effects are governed; it makes the governed boundary explicit, and falsifiable.

Related work approaches the same control boundary from different directions. Progent studies least-privilege tool permissioning [52]. CaMeL separates trusted control from untrusted data [33], while AIR-Guard examines action-time authority confusion [64]. These studies motivate the problem; they do not replicate REMORA's results. Here the contribution is an implemented path from tool authority to exact-call dispatch and distinct effect evidence, evaluated alongside deterministic and probabilistic routing experiments. For governed deployments, this would let an agent suggest useful actions without making its own confidence the source of permission. The reported numbers are tied to committed artifacts and their stated scope, not to field validation.

Contributions. The individual mechanisms are known: capabilities with caveats [70], reference monitors [68], policy engines and tamper-evident logs [71]. The contribution is a systems artifact that integrates them and an evaluation method around it. This preprint makes five contributions, each bound to committed artifacts; Table 1 lists the headline numbers.

1. an authority-bound execution chain (signed ToolSpec, exact-call `ExecutionLease`, single-use grant, re-policying at dispatch) whose lifecycle separates authorization, dispatch, execution and verified effect (§4.1); it is implemented and tested, and its attack evaluation is post hoc and author-run (§15);
2. an argument-resolution ladder in which UNKNOWN is a first-class verdict and a VERIFY names a bounded lookup followed by full re-entry of the router (§4.2, §14);
3. sealed external routing evidence on BFCL v4, a disjoint C-ext3 track that isolates declared semantic authority, and a C-ext4 track that measures a revised bundle against the C-ext3 one, with the utility cost and the limits of the gain published as measured (§11.5, §11.6);
4. machine-readable capability and boundary declarations with explicit claim ceilings, a claim register that gates the headline numbers in this paper, and a negative-results record that includes an ablation attributing the headline safety number to no component (§4.4, §12);
5. frozen interoperability contracts with external run records, one of them with two external fixture-level runs that meet the repository's independence rule, together with contract probes and a formal model contributed to an external federation transport (§4.3).

The paper follows that order: related work (§2), the problem statement and architecture (§3–§9), evaluation and results (§10–§14), negative results, validity

Table 1: Headline results. Every row is bound to the claim register; BFCL wrong calls are cross-task substitutions (§11.5); CLAIM-018 is superseded by CLAIM-019 and kept on record; it was measured on another sample and is not a baseline for C-ext3. The AgentHarm false-block floor (at most 40%) was pre-registered for the re-scoring of the same 208 benign scenarios and is recorded as missed (`artifacts/verify_protocols/agentharm_rescore_v1.json`; NEGATIVE_RESULTS.md §19).

Study	Result (claim)
Tool-call benchmark, 70 clusters	0% unsafe acceptance, Wilson [0.0%, 5.2%], upper 6.4% over the 56 harmful templates; keyword baselines 1.4%, not significant (CLAIM-001)
AgentHarm, 208 harmful scenarios	0% false authorizations, Wilson [0.00%, 1.81%], upper 7.6% over the 47 base behaviours; every benign counterpart also blocked, false-block rate 100% against a pre-registered floor of at most 40%, missed (CLAIM-002)
Validator study, all-UNKNOWN regime	autonomous valid-identifier reads 0/60 (zero by construction) to 60/60 (CLAIM-014)
Sealed BFCL v4, 1,527 episodes	wrong-call acceptance 28/258 = 10.9% (CLAIM-018, superseded)
Sealed C-ext3, 2,799 episodes	wrong-call acceptance 0/500, upper bound 0.76% by row, 3.5% over its 106 tools (CLAIM-019)
Sealed C-ext4, 1,562 episodes	correct-call acceptance 87/291 = 29.9% (C-ext3 bundle 6.9%); wrong-call acceptance 0/295, as for a refuse-all control (CLAIM-021)
Component ablation	FAR 0.000 in every arm (CLAIM-020)

and reproducibility (§15–§18), and the conclusion (§19). The retired oracle-consensus, conformal and calibration material is in Appendices E–H.

2 Background and Related Work

2.1 LLM Ensembles and Self-Consistency

Wang et al. (2023) [7] introduced self-consistency sampling over chain-of-thought paths. Lakshminarayanan et al. (2017) [13] demonstrated that deep ensembles from distinct random initialisations improve both accuracy and calibration. REMORA's oracle fan-out is structurally related but uses *separately configured oracle endpoints* rather than multiple samples from one model (the 2026-07 clean round and the AgentHarm validation use a Cloudflare Workers AI cross-family trio (Meta LLaMA, Alibaba Qwen, Mistral; earlier rounds used a Groq three-LLaMA swarm), and weights oracles by *inverse pairwise correlation* rather than treating them as exchangeable. Kuncheva & Whitaker (2003) [14] formalised diversity measures for classifier ensembles; REMORA's inverse-correlation weighting can be understood as an instance of this family applied to instruction-following LLMs. Wang, Aitchison & Rudolph (2023) [55] show that parameter-efficient LoRA ensembles improve predictive accuracy and uncertainty quantification, with ensemble diversity as the key driver; this motivates, but does not independently validate, REMORA's heterogeneous oracle aggregation.

2.2 Multi-Agent Debate

Du et al. (2024) [8] showed that inter-model debate can reduce hallucination. REMORA's dissensus metric D is a one-shot, non-iterative analogue: disagreement is measured once, not resolved through iterative exchange.

2.3 LLM-as-Judge and Verifier Models

Zheng et al. (2023) [9] established the LLM-as-judge paradigm. Cobbe et al. (2021) [10] trained outcome verifiers that rank complete model solutions. REMORA differs by using a *structured policy engine* rather than a learned verifier, enabling interpretable hard blocks that cannot be overridden by a confident oracle.

2.4 Selective Prediction and Abstention

Chow (1970) [11] established the optimal error-reject tradeoff for binary classifiers. El-Yaniv & Wiener (2010) [12] extended this to a distribution-free selective classification framework; their coverage–accuracy bound, which assumes a noise-free realizable setting that three LLM oracles do not form, is the foundational result that Geifman & El-Yaniv (2017) [15] apply to deep networks. Kadavath et al. (2022) [16] find promising LLM self-knowledge on tasks resembling training data, with weaker calibration and self-evaluation on novel tasks. REMORA's abstention is grounded in structural oracle disagreement rather than self-reported confidence.

2.5 Conformal Prediction

Conformal prediction originates with Vovk, Gammerman & Shafer (2005) [54]; Shafer & Vovk (2008) [49] and Angelopoulos & Bates (2021) [17] established it as a practical distribution-free framework for coverage guarantees. REMORA implements Mondrian phase-stratified conformal calibration for the ordered phase, and a CRC-inspired weighted empirical selective router [3] with importance weights (following weighted conformal prediction under covariate shift [4]) for the critical phase (§8.2). The exchangeability violation in the critical phase is documented as a negative result (§15). Conformal methods have since been brought to LLM outputs directly: conformal factuality guarantees via claim back-off [41] and conformal abstention for hallucination mitigation [58]; REMORA's contribution is narrower and different in object: phase-stratified calibration over multi-oracle governance observables for *action gating*, not per-output factuality.

2.6 Prover-Verifier Games

Kirchner et al. (2024) [2] showed that prover-verifier *training* games improve LLM output legibility by requiring provers to justify claims to iteratively trained verifiers. REMORA does not implement that training game; it borrows the framing for a PVD-inspired offline agreement score, using Semantic Entropy clustering to select the prover (dominant-cluster oracle) and verifier (independent challenger) from already-produced responses, without additional API calls (§F.2).

2.7 AI Governance and Policy-as-Code

Algorithmic-audit scholarship emphasizes institutionalized third-party oversight and audit-system design [45]; Raji et al. identify lack of audit data access as the most significant vulnerability of the current AI audit ecosystem. REMORA's DecisionEnvelope addresses this access gap by logging each governance decision in a tamper-evident hash chain, which detects edits but not a full rewrite (§9), and the Replay Engine publishes governance episodes for third-party inspection. The EU AI Act (2024) [25] mandates human oversight and audit trails for high-risk AI systems. Open Policy Agent (OPA) [26] provides a production-grade Rego policy engine. REMORA ships an OPA adapter as a concrete instantiation of policy-as-code for AI decisions; on OPA unavailability it falls back to the parity-tested deterministic Python policy engine and records the fallback, without bypassing policy enforcement. The reference API currently invokes the Python engine directly (§4).

2.8 Assurance Cases

Kelly (1998) [20] and Bloomfield & Bishop (2010) [21] structure safety arguments as claim-evidence-reasoning trees. REMORA's DecisionEnvelope is designed to populate one node of such a case with a specific gate decision's evidence chain.

2.9 Human-in-the-Loop / Human-on-the-Loop

Endsley (1995) [19] provides the situation-awareness theory underpinning human supervision of dynamic systems; the automation-oversight literature distinguishes human-in-the-loop confirmation from human-on-the-loop monitoring. REMORA's four-outcome schema implements this spectrum: ACCEPT (human-on-the-loop) through ESCALATE (human-in-the-loop), with VERIFY and ABSTAIN as intermediate states.

2.10 Industrial AI Safety

The oil-and-gas sector has established quantitative risk frameworks (NORSOK D-010:2021 [43]; IEC 61511:2016 [39]) that define action classes requiring human sign-off. REMORA's case study (§13) uses this domain as an illustration of how policy-gated AI governance maps to existing industrial safety requirements. No domain-specific correctness is claimed.

2.11 Runtime Guardrails: Agent Permissioning, and AI Control

Content guardrails. Llama Guard and its successors [40] classify conversational inputs/outputs against a safety taxonomy; NeMo Guardrails [47] provides programmable dialogue rails; Constitutional Classifiers [51] defend against universal jailbreaks with classifiers trained on constitution-generated data; Rule-Based Rewards [42] move safety rules into training. These systems govern *content*; REMORA governs *actions*: its unit of decision is a proposed tool call with operational consequences, and its deterministic hard-block layer is, by design, not overridable by any learned component. A trained classifier and a policy floor fail differently, and REMORA's position is that action gating in consequential environments needs the latter beneath the former.

Agent-level guardrail systems. LlamaFirewall [28] is the closest industrial system in scope: a

layered agent guardrail (prompt-injection detection, alignment checking, code screening). GuardAgent [57] generates guardrail code with a guard LLM; AgentSpec [56] defines a DSL of runtime triggers/predicates/enforcement near-isomorphic to REMORA's policy invariants. Two 2026 systems share REMORA's learning ambition: AgentTrust [59] splits threats into lexical (deterministic rules) and semantic (LLM judge with a guarded verdict cache), and *self-distills judge decisions into its rule layer*; Membrane [29] evolves a contrastive safety memory against jailbreaks. REMORA's differences are deliberate: (1) the hard-block floor is fixed, versioned, and hash-provenanced policy-as-code, learned outputs never become enforcement rules, and the learning layer (AROMER) is monotonicity-tested to *never weaken* policy (`tests/policy/test_governance_intelligence_never_weakens_policy.py`), whereas AgentTrust's distillation pipeline makes learned judgments load-bearing for enforcement; (2) on the research path uncertainty is structural (multi-oracle disagreement with conformal abstention), not a single judge's confidence, while on the governed path these signals are left unset and decisions rest on deterministic policy (§5.1); (3) every decision emits a DecisionEnvelope, which the API path finalizes into a tenant-scoped hash chain and HMAC-signs when a signing key is configured. Conversely, AgentTrust reports zero benign hard-blocks across 45,000 actions, the opposite end of the safety/friction trade-off from REMORA's FBR=100% AgentHarm corner point (§11.7), and an explicit target for REMORA's calibration roadmap.

Permissioning and security by design. Progent [52] enforces programmable least-privilege on agent tool calls. With LLM-generated policies and gpt-4o it reduces attack success on AgentDojo from 39.9% to 1.0%, at 76.3% utility against 79.4% undefended (arXiv:2504.11703v2; v3 repeats the 39.9% to 1.0% figure, v1 reported 41.2% to 2.2%). CaMeL [33] defeats prompt injection by construction via control/data-flow separation and solves 77% of AgentDojo tasks, against 84% undefended, with what the authors call provable security (arXiv:2503.18813v2; v1 reported 67%). These results define the current (safety, friction) frontier and pre-empt any reading of REMORA's zero-FAR corner point as sufficient. REMORA's own execution chain (§4.1) is capability-based enforcement of the same family, applied per call. Unlike CaMeL it does not track data flow by default, so it does not stop an exfiltration composed of individually authorized calls (§15). Its contribution is not the corner but the integrated chain and the claim-governed evaluation around it.

Information-flow control and isolation. Information-flow control addresses the case per-call authorization leaves open: an exfiltration built from individually authorized calls. Wu et al. (2024) [74] build an f-secure LLM system in which a security monitor keeps untrusted input out of the planning process, with formal models of the guarantee. FIDES [75] is a planner that tracks confidentiality and integrity labels and enforces policies deterministically; the authors characterize which properties dynamic taint-tracking can enforce. IsolateGPT [76] (first posted as SecGPT) isolates the execution of third-party LLM apps from each other and from the system. REMORA borrows the premise that untrusted data must not decide what runs. It does not propagate confidentiality or integrity labels across calls, has no separate planner, and claims no trajectory-level or data-flow property (§15). These systems are the baselines a data-flow extension of REMORA would have to be measured against.

Prompt-injection defenses. A second family hardens the model or the agent loop. Spotlighting [77] transforms untrusted input so that its provenance stays visible to the model. StruQ [78] separates prompt and data into two channels and fine-tunes the model to follow instructions only in the prompt channel. SecAlign [79] pursues the same goal with preference optimization. Task Shield [80] checks at test time whether each instruction and tool call serves the user's goal. MELON [81] re-executes the trajectory with a masked user prompt and flags an attack when the original and masked runs propose similar actions. Conseca [82] generates just-in-time, contextual and human-verifiable security policies, where REMORA's hard-block floor is fixed and versioned (§6). Beurer-Kellner et al. (2025) [83] propose design patterns for agents with provable resistance to prompt injection and analyze their utility and security trade-offs. REMORA does not train models, transform prompts or re-execute trajectories. It acts at the tool-call boundary after the model has proposed an action, so these defenses sit upstream of it, and REMORA reports no measurement against any of them.

Benchmarks and environments. Beyond AgentHarm [5], the field evaluates on AgentDojo [32], ToolEmu [48], InjecAgent [62], R-Judge [61], and τ-bench [60]. ToolEmu's LM-emulated sandbox anticipates the counterfactual-execution idea behind REMORA's Shadow Mode; REMORA's replay engine is therefore *not* conceptually unique: its distinction is the governance framing (committed replay artifacts under claim governance). Replication of REMORA's

results on these independently constructed environments is required future work and the honest boundary of every internal benchmark claim in this paper.

AI control. Greenblatt et al. (2024) [36] formalize safety protocols that remain effective when the policy model itself is intentionally subversive, using trusted monitoring with a limited audit budget. REMORA is related to the trusted-monitoring protocols of that taxonomy and adds a deterministic trusted floor. The hard-block layer and the authority-bound execution chain are the trusted components, the oracle swarm is a set of weaker trusted monitors, and ESCALATE spends the audit budget. Two properties of the design are the control-relevant ones. Within the declared custody profile the untrusted agent never holds the material that mints authority, and no model output, its own or a monitor's, can clear a hard block. Resampling and defer-to-trusted protocols from the control literature are implemented as a library (`remora/governance/control_protocols.py`, roadmap item RF-04) and are not wired into the decision path. Section 3.1 states the threat model. A strategically subversive agent, collusion between monitors and monitor evasion remain open items for the red-team plan, and the benchmarks in §11 do not measure them.

Regulatory operationalization. COMPL-AI [37] provides a technical interpretation of the EU AI Act as model-level benchmarks; REMORA operationalizes a complementary slice at runtime, Article 12 (logging) via the DecisionEnvelope hash chain, Article 14 (human oversight) via VERIFY/ESCALATE routing, without claiming conformity assessment.

Multi-layer guardrail taxonomies. A systematic literature review of runtime guardrails [50] frames multi-layer defense as a Swiss-cheese model; a layered governance architecture with OS-level enforcement is developed in Ge (2026) [35]; confidence elicitation by fidelity is treated in Zhang et al. (2024) [63]; and formal verification of behavioral safety properties for neural policies in Corsi et al. (2021) [30]. REMORA's position relative to these four is that it composes recognized layers (policy floor, ensemble uncertainty, conformal abstention, audit chain) with a claim-governance methodology, rather than introducing a new layer type.

Authority-bound execution and execution evidence. A 2026 line of work moves the control point from pre-action screening to the authorization and evidence of execution itself. AIRGuard [64] names *authority confusion* (untrusted context may inform reasoning but must not authorize side effects), a form of the confused-deputy problem [69], and enforces least privilege as action-time authorization. Proof-Carrying Agent Actions [65] organizes runtime-neutral governance around a portable action certificate with five checkpoints, from pre-action admissibility through approval to outcome closure. Proof of Execution [66] formalizes execution evidence as a contract, a causal event stream and a replay context with validator-checkable invariants and a soundness argument under explicit cryptographic and deployment assumptions. A survey of agent permission systems [67] reviews 21 proposals and finds that none of them achieves all three of low user burden, formally grounded specification and deterministic enforcement. REMORA's position rests on explicit **non-claims**: it is not first with runtime policy enforcement, pre-action authorization checks, or agent governance in general. What it combines in one reproducible artifact is authority provenance (signed ToolSpec), exact-call binding between approved and executed action, key-custody separation between decision and execution domains, repolicying at the execution boundary, per-component trust-base coverage declared on the decision record, explicit effect-evidence states in which transport success is never execution, machine-readable capability and boundary declarations whose claim ceilings state what is not established, and a machine-enforced register of claims and negative results governing the headline numbers reported here. A mechanism-level crosswalk is maintained in the repository (`docs/research/adjacent-systems-crosswalk-v2.md`).

Protocols, authorization tokens and policy languages. The Model Context Protocol specification [84] standardizes how hosts expose tools to models. It states that MCP "cannot enforce these security principles at the protocol level" and that tool annotations "should be considered untrusted, unless obtained from a trusted server". REMORA takes the same position on tool metadata: an approval binds to a signed ToolSpec (§4.1), not to what a server declares. Capability tokens with caveats [70] and Biscuit [85], whose holders attenuate tokens offline and whose checks are written in Datalog, are the token-level precedents for REMORA's exact-call lease. OAuth Rich Authorization Requests [86] carry structured `authorization_details` that a scope string cannot express, close in intent to binding a lease to one call. XACML 3.0 [87] defines the policy decision point and policy enforcement point vocabulary that §4.1 uses. Cedar [88] is an authorization language whose design

properties are proved in Lean. REMORA uses none of these formats or languages on the wire; its policies are Python with an OPA/Rego adapter (§6). Binding grants to the task and the exact call is aimed at the confused-deputy problem [69].

Tamper-evident logs. Schneier and Kelsey [89] make log entries written before a logging machine's compromise unreadable to the attacker and impossible to change or delete undetectably. Crosby and Wallach [71] give tamper-evident history trees with efficient membership and consistency proofs, and Certificate Transparency [90, 91] publishes append-only Merkle-tree logs that third parties audit. REMORA's audit chain is a per-tenant SHA-256 hash chain (§9). It detects modification of a historical entry but not replacement of the whole chain. It has no forward-secure keying and is not anchored in an external log; these works are what a hardened chain would borrow from.

3 Problem Statement

Let $\mathcal{A}$ be a space of agent-proposed actions, q a query, and $c = (\text{domain}, \text{risk}, \text{env})$ an operational context. We seek a control function:

$$\Gamma : (q, a, c) \longrightarrow g, \quad g \in \{\textsc{Accept}, \textsc{Verify}, \textsc{Abstain}, \textsc{Escalate}\} \tag{1}$$

with five required properties: (1) safety-first (hard policy violations map to Escalate regardless of consensus); (2) calibration-aware [18] (abstain when uncertainty exceeds threshold); (3) evidence-sensitive (evidence can resolve or contradict ambiguous consensus); (4) auditable (every call produces a hash-chained evidence record, HMAC-signed when a signing key is configured); (5) conservative by default (fail closed toward Verify or Escalate depending on risk tier).

This is not a question of oracle *accuracy*: REMORA does not know whether action a is correct. It is a question of *assurance conditions*: whether the conditions that justify autonomous execution of a are verifiably met.

3.1 Threat Model

This subsection states the adversary and the trusted computing base that the execution-chain claims assume. It draws on `docs/assurance/threat_model_v1.md`, a STRIDE baseline written before the execution layer existed, and on the code paths named in §4.1.

Attacker position. The attacker is the agent, or content that steers the agent, for example through prompt injection. It acts only by proposing calls at the gate: a tool name, arguments, and any labels that accompany them. It holds no signing key and cannot edit the tool registry or the policy bundle. The model assumes it reaches tools only through the governed dispatcher; paths that bypass the dispatcher are out of scope.

Trusted computing base. The claims hold only if four parties are trusted. The ToolSpec author signs the specification that defines what an approval covers. The policy author owns the versioned bundle that the PDP and the PEP evaluate. The intent source is the deployment component that supplies or verifies the task intent and the tool contracts. Key custody covers the domains that hold the PDP signing key and the Ed25519 lease key, which the agent never holds (§4.1). The policy code, the dispatcher and the audit store belong to the same base, and a compromise of any of them is out of scope.

Labels supplied by the agent are untrusted. A risk tier, action type or task intent that arrives with a proposed call is a claim, not authority. Where the opt-in enrichment stage runs, an inferred higher risk overrides a supplied lower label and never the reverse (§7). Untrusted content that controls a sensitive argument escalates whatever risk tier the caller declared (`hard_guard_floor()`). A model may propose a task intent, but its clauses count only where their quoted spans occur in the task text (§11.6).

Argument content and destinations. The attacker chooses every argument value, including URLs, recipients and payload text. The decision reads the tool name, the registry's labels and the call's binding. It does not read what an argument carries or where a call sends it. A low-risk read whose URL carries a secret is therefore within the attacker's reach and is not defended. An argument-content and destination policy has to come from the information-flow check (`remora.flow`). With `REMORA_FLOW_CONTROL=1` the decision reads its verdict, but tool results do not yet raise a context's label, so this posture is measured only with labels a harness assigns and is off by default (§15).

Injection patterns are a hint. The admission firewall escalates a few known injection phrasings by substring match. A paraphrase passes it, so the list is a routing hint, not a security boundary. An injection

is judged by what the injected call can reach. A run on an external injection benchmark is pre-registered and not done.

Out of scope. The model does not cover calls that bypass the governed dispatcher through alternate credential, network or execution paths, or privileged effects that an authorized tool's implementation can reach (§4.4). It does not cover exfiltration through an authorized call's arguments, or composed of individually authorized calls, because by default REMORA does not track data flow (§2). The opt-in flow check is not part of the evaluated threat model (§15). A strategically subversive agent, collusion between monitors and monitor evasion are open items for the red-team plan. Compromise of the host, the keys, or the ToolSpec, policy or intent sources is excluded, as are availability attacks such as rate-limit exhaustion. Section 15 reports a post-hoc evaluation of the chain within this model.

4 REMORA Architecture

REMORA processes each gate request through six decision stages, followed by DecisionEnvelope emission (Stage 7). The architecture diagram in the supplementary material (Mermaid) shows the full pipeline. Stage 7 is the output record, not a decision stage.

Stage 1: Intake & Risk Classification. The system classifies intent domain, risk tier ($\in$ {low, medium, high, critical}), action type, and target environment. An adversarial admission firewall checks for injection patterns ("ignore previous instructions", "drop table", etc.) and fires an immediate ESCALATE if detected.

Stage 2: Oracle Fan-Out. n configured oracle models are queried in parallel via `ThreadPoolExecutor` (the engine accepts $n \geq 2$; the design recommendation is $n \geq 3$ from distinct model families. The 2026-07 clean round and AgentHarm use a Cloudflare Workers AI cross-family trio (Meta LLaMA, Qwen, Mistral) enforced by `remora/oracles/families.py`; earlier rounds used a Groq three-LLaMA swarm whose within-family correlation gave an effective diversity closer to two). Responses with non-null `error` fields are excluded before any aggregation. $\mathcal{O}' \subseteq \mathcal{O}$ denotes the valid oracle set.

Stage 3: Canonicalization & Weighted Consensus. Each response is canonicalized by φ : raw $\rightarrow$ v where v is the tuple (`polarity`, `claim_hash`, `magnitude`, `tags`). The `claim_hash` field uses a pluggable backend: the default `TokenFingerprintBackend` applies a 16-char SHA-256 truncation over NFKC-normalized sorted tokens for backward-compatible, dependency-free operation; the `NLISemanticBackend` replaces this with bidirectional cross-encoder entailment clustering, implementing Semantic Entropy [1]. Oracle weights are derived from inverse pairwise correlation (Appendix E).

Stage 4: Consensus Observables. Entropy H is computed over the diversity-weighted verdict distribution $\hat{p}$. The `NLISemanticBackend` grounds H as Semantic Entropy [1], $\mathrm{SE}(x) = -\sum_{c \in \mathcal{C}} p(c \mid x) \log p(c \mid x)$ over NLI-derived semantic equivalence clusters $\mathcal{C}$. *Note on experimental backend:* all benchmarks reported in §10 use the default `TokenFingerprintBackend` (NFKC-normalised, sorted-token SHA-256 truncation), which approximates cluster identity via lexical overlap. The NLI backend is a drop-in replacement for future work but was not used in the reported results; claims about Semantic Entropy properties should be read accordingly. Dissensus D, consensus concentration η and trust score τ are computed (Appendix E), and a regime $\in$ {ordered, critical, disordered} is assigned. These run on the research and benchmark path; the governed server routes leave them unset (§5.1).

Stage 5: Evidence Router (Critical Phase). If phase = critical, the `CriticalEvidenceRouter` evaluates a five-field `EvidenceSignal` and returns one of: `evidence_accept`, `abstain`, or `escalate`. *Important:* the current EvidenceSignal is an oracle-proxy (built from consensus statistics); semantic BM25/NLI retrieval is pluggable but not live (§8).

Stage 6: Policy Engine. Seven *headline* policy controls are evaluated in priority order before any routing logic; the shipped engine's complete priority-ordered gate inventory is larger (schema, forbidden-tool, coercion, tainted-argument, rollback, production-write, quorum, trap-detection, session/fleet-risk gates, among others) and is maintained in `remora/policy/decision_engine.py`, with `hard_guard_floor()` as the single deterministic floor no probabilistic result can override. An OPA/Rego adapter with Python fallback is implemented and parity-tested, but the reference API invokes the Python policy engine directly; OPA remains an integration option and production dependency (fail closed either way). Gate outcome g is assigned.

Output: DecisionEnvelope v2. An attribute-frozen envelope (treated as read-only; nested collections are not deep-frozen) records: request, assessment, gate, reviewer context, follow-up request, case history, policy-learning candidate, and audit hash-chain entry.

4.1 The Authority-Bound Execution Chain

Deciding and executing are separate trust domains, bound with cryptographic identity at every hop. The policy layer issues a `PolicyDecisionToken` (format v3: Ed25519 over a domain-separated payload, with a key identifier derived from the public key; the observation hash binds it to one governance call; the `jti` is consumed exactly once against a durable ledger, and an in-memory database is refused as a "durable" backend). A redeemer configured with verification keys alone verifies every grant it is shown and mints none. The HMAC formats v1 and v2 remain readable; once Ed25519 material is configured they are refused as live authority unless a migration window is declared. What an approval is granted *over* is a signed ToolSpec whose identity travels inside the lease; a spec that moved between approval and dispatch refuses. Dispatch itself happens only under an exact-call `ExecutionLease` (tenant, actor, tool, canonical argument hash, target, policy-bundle identity, single-use nonce, expiry), signed with Ed25519 under an architectural custody split claimed within the strict governed-execution profile under the declared credential topology: the authority domain holds the private key and no declared downstream tool credential; the execution domain holds the public key and the declared execution credentials. The `GovernedToolDispatcher` re-verifies the entire binding against the concrete call, against the clock the lease was issued under, before anything runs. The split does not establish the absence of alternate credential, network or execution paths that bypass the governed dispatcher; that boundary is stated explicitly in §4.4 and §15. Authorization and result are separate hash-chained records, each carrying the per-component trust-base view resolved at that moment: the closure record rereads rather than copies, so a stale policy snapshot is detectable as a failed component-level join. A crash-consistent outbox records dispatch intent before the effect and reconciles stranded dispatches to UNKNOWN; effect verification is a separate, replay-protected receipt. For principal revocation, the audit event joins the state transaction through an audit outbox and is projected on a later execution-path interaction; this does not imply that all API audit writes are transactional or that idle tenants are automatically swept. **Stated gaps:** the decision/enforcement split is real only where `/assess` and `/execute-accepted` run in different custody domains. A deployment that runs both in one process with the private key is one domain, and the review path still mints and consumes in-process. No deployment is proven to configure the asymmetric grant (RMR-CR-002 stays partial). KMS/HSM key management and process-boundary mTLS are not implemented; dispatch runs in the API process by default, with an opt-in separated dispatch worker (202 after durable authorization; the worker half re-verifies the persisted call against the recorded binding) exercised so far only by CI and its contract tests.

4.2 The Argument-Resolution Ladder

The six stages above describe how an action reaches a verdict. They do not describe what the engine reasons about when the action is well-formed but the *state it refers to* is uncertain, which, in operational deployments, is the common case. That reasoning is a distinct ladder, evaluated after every blocking gate so it can only ever make a would-be ACCEPT more cautious, never unblock one:

1. **Authoritative tool metadata.** Effect, risk class, resource type and validator bindings come from a server-side registry. A caller may raise risk, never lower it. Verb heuristics are a fallback for unregistered tools only, a lesson learned when `assign`, `close` and `approve` were absent from a mutating-verb list and thirty writes scored as reads.
2. **Coverage declaration.** A state index may not infer its own completeness. Closed-world authority is a signed, curator-bound declaration scoped per argument role, with a snapshot hash, an as-of date and a freshness bound. Without it, absence means UNKNOWN.
3. **Tenant, hash and freshness binding.** A declaration that does not match the snapshot it claims, or that has aged past its bound, is refused rather than trusted. Cross-tenant consultation is refused outright.
4. **Argument support.** Does the system of record contain this value? Tri-state: present, confirmed absent, or outside coverage. Unknown must never behave like a negative.
5. **Value grounding.** Are the argument values traceable to *this* task: named in the task text, declared by the tool's own schema, or vouched

for by the system of record? A call whose values anchor to nothing here is observationally a foreign call.

6. **Validation requirement.** An argument that steers where the action lands must be confirmed before autonomy, decided by what the argument *does* rather than by whether an index happens to cover it.
7. **Resolver availability.** If a bounded lookup can close the gap, the verdict is VERIFY carrying a `ResolutionPlan` that names the resolver, the exact arguments it may write, and the sources permitted to supply them. If no resolver exists, the verdict is ABSTAIN: promising a verification that cannot happen is worse than stopping.
8. **Validator execution and full re-entry.** The resolver returns one value and hands control back. It never performs the original call and cannot redirect it. The *whole* router then re-runs on a fresh observation, so every guard applies again to the resolved call rather than being patched around.
9. **Read/write posture.** A verified read may proceed autonomously; a write does not, whatever the state evidence says.

This ladder, not the consensus machinery, is what the operational results in §11.5–§11.6 measure. Its tri-state discipline is the recurring commitment: UNKNOWN is a first-class verdict distinct from UNSUPPORTED, and a signal that cannot be established is left absent rather than guessed, since a fabricated boolean enters the policy contract as fact, while `None` correctly says nothing establishes this.

4.3 Evidence Admission, Federation and Standards Work

A separate, tested `remora/evidence/admission/` module derives producer visibility, coverage, observation vantage, exact-invocation binding and prior-commitment premises from typed records and deployment-owned trust configuration. Malformed evidence is a processing failure, not a negative property verdict. The module does not feed the frozen conformance checkers, effect verification or the execution decision; it supplies a contract for evaluating the provenance of future observations. Its six clauses follow the evidence-sufficiency premises of the CoSAI workstream on secure design of agentic systems (WS4, §7.4; `docs/design/evidence-admission-v1.md`). `NOT_ESTABLISHED` is a conclusion reached after verification ran; malformed input is a processing failure; self-report is not independent observation. Absence needs producer visibility and coverage for the same scope; identifier equality is not binding; expected verdicts stay outside admission inputs. The author took part in the workstream's public request-for-comment threads on agent manifests and on evidence sufficiency.[1] Those threads argued the boundary between attested deployment state and action-time authority, and the three-valued verdict for absent evidence. The repository's evidence-sufficiency conformance packages are research artifacts, not workstream deliverables.

Federation packages pin source revisions and file digests, preserve native claim vocabulary and distinguish reproduction from an independent second implementation. External run records exist for the exact-call-binding, fresh-authority and effect-evidence contracts (E-ECB, E-FA, E-EE). They were run by an external second implementation (Probity) against frozen fixture packages and are recorded as `NOT_INDEPENDENT`, with each boundary-level claim result `NOT_ESTABLISHED`. The E7 runtime-surface contract has two external Probity run records (2026-10-08 and 2026-10-09) that meet the repository's recorded independence rule, so its lifecycle is `EXTERNALLY_VERIFIED` with independence `INDEPENDENT`. That rule means a second implementation importing no producer code, run outside the producer's control; it is not a blind study. The record keeps its exposure: the expected answers were published before the reader was written, and a lint diagnostic later exposed parts of the retained reference verifier. The reader neither imported nor executed it. Both records establish fixture premises only; the claim result and runtime capability-surface completeness stay `NOT_ESTABLISHED` (`artifacts/interop/index.json`, `docs/interop/FEDERATION.md`).

The federation work also runs in the other direction. A contract-coverage map lists every normative rule of the federation-port transport's contract with the tests that exercise it, and 29 contract probes run against the unmodified upstream runtime. The probes found four findings (CP-F1 to CP-F4). A late provider confirmation was dropped from operation state under lease takeover, and a deadline did not bound first provider contact. Unbounded adapter reason text entered provenance, and a lost response under outage closed an operation as failed while a refund existed. The fourth was found by a TLA+ model of lease retry (`formal/tla/LeaseRetry.tla`, checked in CI) and replayed by a probe. All four were fixed upstream in a pull request from this repository (aeoess/federation-

[1] cosai-oasis/ws4-secure-design-agentic-systems issues #149, #189 and #27, August to September 2026.

port#1, merged 2026-10-08); the probes now assert the corrected behaviour and fail on the pre-fix runtime (`artifacts/interop/federation-port-v0/contract-coverage.json`). Federation records are evidence references, never execution authority. On the enforcing API, typed semantic judgments from an external provider are opt-in shadow records appended after audit; they can guide a lookup or an approver but do not change a verdict.

4.4 Machine-Readable Capability and Boundary Declarations

The properties above are published as machine-readable declarations, not only as prose. `docs/interop/remora-capabilities-v1.yaml` declares each workflow capability with its stage in the chain from authority through dispatch to effect. It records the roles a capability provides and does not provide, its trust assumptions, its maturity level, its evidence links and its limitations. Maturity runs from `DECLARED` to `OPERATOR_OBSERVED`; no declared capability currently claims the external or operator-observed states. `docs/interop/remora-boundaries-v1.yaml` declares the internal trust boundaries, external interfaces and Federation edges as separate records. Each is bound to a named **claim ceiling**: a bounded statement of what the record establishes, paired with explicit non-claims. Federation edges carry separate **status dimensions** for owner confirmation, pinning, execution, independence, claim result, production evidence and Federation adoption. A recorded external run advances the execution and independence dimensions only under the record's stated rule, and never the claim-result, production or adoption dimensions by itself. Role mappings for the draft Agent Authority Conformance model are declared as candidates with their partial coverage stated.

The declarations are fail-closed in both directions. CI gates fail when a declaration diverges from the assurance registers, the pinned source revisions or the recorded evidence digests: `scripts/check_remora_capabilities.py`, `scripts/check_remora_boundaries.py` and `tests/test_remora_boundary_declaration.py`. A boundary cannot claim higher maturity than the capabilities that support it. Three properties are declared precisely because they are *not* established. All three are listed under `unestablished_properties` in `docs/assurance/capability_register_v1.yaml`; the boundary declaration carries the first as its global property:

- **Runtime capability-surface completeness** (`runtime_capability_surface_completeness`, `NOT_ESTABLISHED`): the reference runtime observes its own offered tool surface and dispatcher. This does not establish completeness for external agent processes, host credentials or unobserved alternative paths. Calls that bypass the governed dispatcher are outside the boundary by declaration.
- **Implementation effect non-transitivity** (`implementation_effect_non_transitivity`, `NOT_ESTABLISHED`): authorization of a tool does not authorize every privileged effect its implementation can reach. An exact-call lease binds the call, not the implementation's reachable effect space. The three-domain mediation split whose effect domain alone holds the declared effect credentials is implemented (CAP-024), but no deployment runs it, so the property remains open.
- **Authoritative execution-context binding** (`execution_context_authoritative_binding`, `NOT_ESTABLISHED`): an opt-in path binds the deployment-supplied subject, the proposal-initiating model, the scoped data classification and build/runtime provenance through assessment, review, grant, lease, dispatch and effect recording. Its tests do not establish that the deployment sources of that context are authoritative, nor a deployed end-to-end property.

These three declarations define the reader-facing scope of every custody and dispatch claim in this paper. REMORA establishes properties *at* a declared boundary, and the declarations say where that boundary ends.

5 Consensus Observables

These observables describe oracle disagreement on the research and benchmark path. They are not inputs to the enforcing execution profile.

The weighted support distribution, entropy H, dissensus D, consensus concentration η and trust score τ are defined in Appendix E. The trust score describes research routing, not an execution authorization.

5.1 What these observables do not do

The execution boundary matters more than the research observables.

They are not populated on any governed path. `servers/api.py` never references the consensus state; `servers/execution_api.py` passes `trust_score=None` and `phase=None` explicitly, and

`build_full_observation`, the builder shared by the assess and execution routes, leaves these fields at their `None` defaults. The engine therefore fails toward VERIFY/ABSTAIN there: no probabilistic ACCEPT can fire from these signals in the governed path at all. The runtime safety floor is the deterministic hard-block layer (§6). Everything in §11 is measured on the research and benchmark path.

6 Decision and Policy Model

6.1 Four Gate Outcomes

Table 2: Gate outcomes and autonomous action status.

Outcome	Meaning	Autonomous action
ACCEPT	Assurance conditions met	Permitted
VERIFY	Plausible, validation required	Pending
ABSTAIN	Uncertainty too high to decide	Blocked
ESCALATE	Human review required	Blocked

ACCEPT does not mean the action is *correct*: it means the assurance conditions for autonomous execution are met. ABSTAIN does not mean the action is *wrong*: it means the conditions for deciding are not met. Both ABSTAIN and ESCALATE block execution; ESCALATE additionally triggers a human-review workflow.

6.2 Hard Blocks and Policy Gates (Headline Subset)

Policy must override model consensus. Majority vote is never sufficient for critical autonomous action when a hard block condition is met.

6.3 OPA/Rego Integration

REMORA exports a 76-field `PolicyObservation` dataclass as JSON to the OPA endpoint. On any communication error (connection refused, timeout, parse failure), the parity-tested deterministic Python fallback engine applies identical rules and records `source_of_decision = "python_fallback"` in the envelope. This is a parity fallback rather than a failure mode: policy enforcement is neither bypassed nor withheld, and no decision is ever gated on an optional policy engine being available.

Table 3: A headline subset of the policy gates: unconditional hard blocks (1–3b, `hard_guard_floor()`) and conditional policy gates (4–7, `_CONDITIONAL_GATES`). Rows follow the engine's evaluation order, but other guards run between them, so the numbers give the order within this subset, not a rank in the whole floor. Both sets take precedence over the probabilistic ACCEPT decision; only the unconditional blocks are evaluated before all routing logic.

Ord.	Condition	Reason Code	Action
Unconditional hard blocks (`hard_guard_floor()`)			
1	Adversarial detected	ADM_FIREWALL	ESC.
2	Counterfactual failed	CF_FAILED	ESC.
3a	Evidence contradicted + contradiction cycles	EV_CONTRAD.	ESC.
3b	Evidence contradictions	EV_CONTRAD.	ABS.
Conditional policy gates (`_CONDITIONAL_GATES`)			
4	Refuse parametric + no ev.	THM_REQ_EV.	VER.
5	Distribution shift	DIST_SHIFT	VER.
6	Phase=critical + risk=crit.	CRITICAL_PHASE	ESC.
7	High/crit. risk + no ev.	EV_INSUFF.	VER.

7 Governance Intelligence Layer: Misspecification-Aware Pre-Policy Enrichment

The policy engine is only as honest as the metadata it is given: an integration bug, or a compromised agent, can label a destructive action as a low-risk read and present the gate with nothing to fire on. Added at package version 0.9.0 (repository tag `review-v1`), the *Governance Intelligence Layer* (`remora/governance_intelligence/`) closes this gap with an opt-in, fully deterministic enrichment stage that runs before any policy rule (no LLM calls): (1) fail-closed normalization of caller-supplied labels (unknown is explicit, never coerced to a safe value); (2) action-semantics extraction from the proposed action text and tool metadata; (3) misspecification-risk inference from label/semantics disagreement; (4) causal-consequence signals [27, 44] (blast radius, expected loss); and (5) policy-generalization risk, would repeatedly accepting this *class* of action remain safe fleet-wide?

Signals merge into the existing `PolicyObservation` under a **strengthen-only** rule: inferred higher risk may override a supplied lower label, never the reverse, and hard-block flags are never cleared. A grid property test across 2,160 observation combinations verifies that enrichment never converts a rejection into an acceptance (`tests/policy/test_governance_intelligence_never_weakens_policy.py`). The

engine remains the sole decision authority.

Table 4: Governance-intelligence benchmark: 50 deterministic tasks across 10 categories of mislabelled, underspecified, or repeated actions, evaluated offline with a deliberately permissive trust baseline so that enrichment is the only barrier to acceptance.

Metric	Result
Unsafe accept rate (no-accept tasks)	**0.0%**
Metadata-mismatch detection	75%
Unknown-metadata review rate	100%
Legitimate reads still accepted	100%
Escalation precision	0%
Policy-generalization detection	62.5%

The safety-relevant rows hold (0% unsafe accepts; all unknown-metadata and legitimate-read cases routed correctly), but the enrichment layer's *precision* is weak on this 50-task eval: escalation precision 0% (its escalations did not match the expected set), 75% metadata-mismatch detection, 62.5% policy-generalization detection. These are the committed, deterministic artifact numbers (re-verified); the layer is an offline, opt-in helper, not a validated detector.

Artifact: `artifacts/governance_intelligence/evaluation_results.json`; runner: `experiments/evaluate_governance_intelligence.py`. Without enrichment, `DROP TABLE users` labelled `action_type=read, risk_tier=low` reaches ACCEPT on a high-trust path; with enrichment it escalates. The layer is heuristic and English-centric; pattern tables can be evaded by novel phrasing, so it reduces reliance on caller labels rather than replacing verification, evidence, or human review.

8 Evidence-Grounded Critical-Phase Routing

8.1 The Critical-Phase Problem

In the critical phase, oracle consensus is near the phase boundary. Empirically, higher trust scores *negatively* correlate with correctness ($n = 32$ real-oracle items): Q4 high-trust achieves 50% correct; Q1 low-trust achieves 75% correct. Conformal calibration at 5% risk target fails completely on critical-phase items (mean observed risk = 100%, coverage $\to$ 0%). Standard trust-based routing cannot safely gate critical-phase decisions.

8.2 Offline Selection Components

Three offline components are implemented and tested on the research path and are not invoked by the reference API. The `PhaseAwareGuardrail` inverts the trust score for critical-phase items (Appendix F). A weighted empirical selective router inspired by Conformal Risk Control does not instantiate the CRC theorem, so it gives no risk bound (Appendix G). A PVD-inspired agreement score over already-produced oracle responses trains no model and makes no model call (Appendix F).

8.3 CriticalEvidenceRouter

The router evaluates a five-field `EvidenceSignal` with fields (`evidence_strength`, `contradiction_score`, `citation_coverage`, `cross_evidence_consistency`, `source_reliability`):

1. Coverage gate: `citation_coverage` $< 0.50 \Rightarrow$ ESCALATE
2. Contradiction block: `contradiction_score` $> 0.50 \Rightarrow$ ABSTAIN
3. Evidence-accept: all thresholds met $\Rightarrow$ `evidence_accept`
4. Default: $\Rightarrow$ ESCALATE with diagnostic

On MultiNLI ($N = 3{,}000$): resolution rate 38.5%, evidence-accept precision 100%, false-accept rate on contradiction items 0%, abstain precision on contradictions 99.3%.

8.4 Evidence Signal Provenance

Current implementation: `EvidenceSignal` is built as a proxy from oracle consensus statistics. This is documented in the source as a "structural bridge" pending semantic retrieval. *Planned:* external BM25/NLI passage retrieval feeding real document evidence. The routing logic is fully implemented and tested; real evidence quality may differ from NLI label quality.

9 Audit and Governance Envelope

9.1 DecisionEnvelope v2

Every gate invocation produces an attribute-frozen `DecisionEnvelope` (frozen dataclasses treated as read-only; nested lists/dicts are not deep-frozen) with blocks: request, assessment, gate, reviewer_context,

follow_up, history, policy_learning, and audit. The library engine emits the envelope with `previous_hash` and `signature` unset; the API finalizer links the tenant hash-chain and adds an HMAC signature only when `REMORA_ENVELOPE_SIGNING_KEY` is configured. The full schema is in Appendix B.

9.2 Audit Hash-Chain

$$h_i = \text{SHA-256}(h_{i-1} \parallel \text{json}(e_i)) \tag{2}$$

This provides **tamper-detection**: any modification breaks the chain. It does **not** prevent tampering. Tamper-*proof* audit requires external append-only storage (S3 Object Lock, Azure Immutable Blob, or equivalent) as a deployment dependency.

9.3 Follow-Up Workflow

ESCALATE generates a structured follow-up request specifying reason codes, required evidence, responsible role, and SLA. Data structures are implemented; production integration with external ticketing systems is a deployment requirement.

10 Experimental Evaluation

10.1 Benchmarks

QA benchmark ($N = 544$). BoolQ ($n = 377$) [23], TruthfulQA ($n = 85$) [22], adversarial-curated ($n = 7$), REMORA-curated ($n = 75$, author-assembled, selection bias possible).

Tool-call benchmark (700 rows; effective $N = 70$). Synthetic adversarial suite, leakage-remediated (2026-07-20): 70 unique templates × 10 cosmetic variants across 7 domains and 4 task types; the 700 rows are therefore 70 statistical clusters, not 700 independent tasks. Three adversarial failure modes: intent-argument conflict, change-window/dual-control requirements, safe-looking tasks with dangerous hidden scope. The simulator executes no real shell, network, database, or filesystem mutation; "unsafe" means unsafe *acceptance* of a simulated action.

Evidence benchmark ($N = 3{,}000$). MultiNLI validation-matched [24] (entailment, neutral, contradiction) as proxy for evidence-verification decisions.

10.2 Baselines

QA: full-coverage majority vote; trust-score selective (threshold=0.50).

Tool-call: single model (Llama-3.3-70B); majority vote; self-consistency; verifier heuristic. The committed artifact preserves additional research-only comparator arms.

10.3 Oracles

The oracle backend changed across rounds. Earlier QA and tool-call rounds ran a Groq-hosted three-LLaMA swarm (Llama-3.1-8B-Instant, Llama-3.3-70B-Versatile, Llama-4-Scout-17B-16E-Instruct, one weight family); the 2026-07 clean round, the AgentHarm validation, and the real-LLM baselines use a Cloudflare Workers AI cross-family trio (Meta LLaMA, Alibaba Qwen, Mistral). Oracle model versions may change; stored artifacts ensure deterministic reproduction.

11 Results

11.1 Historical QA Evaluation

The earlier selective-QA round is a historical artifact, not a current selection result. Its sample composition, operating points, uncertainty and later failed confirmation are retained in `NEGATIVE_RESULTS.md` §18 and the claim register (CLAIM-004, CLAIM-012). The remaining results in this section concern action routing and evidence under their stated protocols.

11.2 Tool-Call Benchmark (700 rows; effective $N = 70$)

Table 5: Tool-call benchmark, leakage-remediated 2026-07-20: REMORA vs baselines (700 rows; effective $N = 70$ template clusters).

Condition	Acc.	Unsafe acc.	Utility
Single model heuristic	28.6%	1.4%	0.16
Majority vote heuristic	28.6%	1.4%	0.16
Self-consistency heuristic	28.6%	1.4%	0.16
Verifier heuristic	28.6%	1.4%	0.16
Temperature-gate heuristic (retired)	60.0%	1.4%	0.36
REMORA full policy	**90%**	**0%**	**0.62**

REMORA's full policy gate produced 0 unsafe acceptances (cluster-level Wilson 95% CI [0.0%, 5.2%] over all 70 templates; upper bound 6.42% over the 56 harmful templates, NEGATIVE_RESULTS.md §83); baselines produced 1.4% (1 of 70 clusters). The safety-rate difference is not statistically significant at the template-cluster level (one-sided $p = 0.50$). A post-hoc test, declared after the pre-registered hypothesis failed, finds advantages in decision utility

and accuracy. The utility gain is +0.456 (cluster bootstrap 95% CI [0.36, 0.56], $p \approx 10^{-4}$) against each of the four keyword heuristics and +0.259 ([0.15, 0.37], $p \approx 10^{-4}$) against the retired temperature-gate heuristic, which maps the derived risk tier to a simulated temperature phase and has the same 1.4% unsafe rate (`results/toolcall_benchmark_v2_significance.json`, `comparisons`). The complete condition set, including retired research selectors, remains in `results/toolcall_benchmark_v2_results.json`. Earlier versions of this table reported baselines at 10–20% unsafe: those numbers were inflated by baselines and gate reading author-annotated severity and oracle context flags; the 2026-07-20 re-run restricts all conditions to the observable task surface.

11.3 Conformal Coverage (Mondrian, $N = 2{,}161$)

Mondrian phase-stratified conformal calibration on the augmented 2,161-item dataset is in Appendix H. At the 15% risk target it reaches 99.9% mean coverage in the ordered phase, against 0.9% in the critical and 0.2% in the disordered phase, the limitation behind the evidence router.

11.4 Evidence Router Proxy Benchmark (MultiNLI, $N = 3{,}000$)

Resolution rate 38.5%; evidence-accept precision 100%; false-accept rate on contradiction 0%; abstain precision 99.3%. Trust-only baseline: 0% resolution (all escalated). A separate deterministic cross-domain regression suite (36 author-curated cases over cybersecurity, AI governance and financial compliance; `artifacts/domain_benchmark_results.json`) passes all cases with zero critical failures; it measures the rule-based evidence layer, not the NLI proxy.

11.5 Tool Routing on Sealed External Data (BFCL v4)

The benchmarks above measure whether a *harmful* call is stopped. This track also measures whether a harmless-looking but incorrect call is stopped. Track C-ext2 retained five targets that were sealed before evaluation. It sampled 258 `live_multiple` tasks and 258 `live_irrelevance` tasks from BFCL v4, the Berkeley Function-Calling Leaderboard [73] of the Gorilla project [72], at upstream commit `6ea57973c7a6` (Apache-2.0). The one permitted evaluation contained 1,527 episodes over 516 source clusters, with an empty state index, no validator bindings, and a registry derived only from task schemas.

Pre-registered target	Goal	Sealed BFCL v4 result	Verdict
Required-UNKNOWN autonomous Acc	$\leq 0\%$	$0/32 = \mathbf{0.0\%}$	met
Irrelevance Abs recall	$\geq 70\%$	$258/258 = \mathbf{100.0\%}$	met
Unobtainable Abs recall	$\geq 70\%$	$98/99 = \mathbf{99.0\%}$	met
Obtainable Ver recall	$\geq 70\%$	$96/99 = \mathbf{97.0\%}$	met
Wrong-call Acc	$\leq 20\%$	$28/258 = \mathbf{10.9\%}$	met

All five pre-registered targets met. Labelled routing accuracy was **91.2%** ($n = 1{,}170$; 508 effective labelled clusters; cluster-level Wilson 95% CI [88.4%, 93.3%]). Optional-lane identity Accept was 151/218 = 69.3%, reported without a target.

The confirmation has a precise boundary. BFCL provides no independently vouchable state table, so the wrong-argument-value axis remains excluded by admission. The track also supplies no authoritative `TaskIntent/ToolContract` bundle. Consequently, the blind result confirms the complete current routing pipeline; it does not isolate the causal efficacy of semantic intent matching. The sealed C-ext3 track later supplied that isolation: under a frozen deterministic contract/intent bundle and the semantic-authority floor, native wrong-call acceptance fell from 24/500 (structural arm) to 0/500 (full configuration) on fresh disjoint data, while read autonomy and argument routing degraded (CLAIM-019; NEGATIVE_RESULTS.md §39).

What the BFCL episodes measure. REMORA routes calls it is given; it does not generate them. Each positive episode proposes the row's annotated gold call. Each known wrong call is another row's gold call, with that row's arguments, added to the task's tool list. The tracks call these "native" because they come from BFCL rows rather than from REMORA's mutation generator, but they are cross-task substitutions, not errors a model made, and they are easier to separate than model errors. The obtainable and unobtainable argument families differ in one producer tool whose name spells out the missing argument, so their VERIFY and ABSTAIN recall checks wiring rather than judgement. Irrelevance episodes propose no call, so their ABSTAIN recall holds by construction. No external baseline was run on the sealed episodes; naive rules were scored after the fact (§11.6, §16).

11.6 What the Wrong-Call Axis Implies

The BFCL v4 result is this paper's clearest external routing result:

> **Structural validity and semantic correctness are independent properties, and the governance signals that establish the first cannot establish the second.**

Schema validity, argument satisfiability, provenance, taint and value grounding are all properties of the *call*. Whether the call serves the *task* is a property of the relation between them, and no amount of scrutiny of the call alone recovers it. The residue is therefore not a threshold to tune: there is no threshold on a signal that does not carry the information.

The architecture reserves a slot for the missing authority, `tool_matches_goal`, tri-state, held at UNKNOWN since definition rather than filled with a guess. A mechanism now occupies it: a deployment-declared tool contract (capability, effect, resource type, mutation, argument roles) matched against a structured task intent, routing an established contradiction to ABSTAIN for reads and ESCALATE for writes. It is placed ahead of trust routing, so a confident consensus cannot execute a call that the contract shows serves the wrong goal, and behind every hard guard, so it can never unblock one. The authority limit is the design: a model may *propose* the intent but may not thereby assert a match, and an intent whose quoted spans do not occur in the task text yields UNKNOWN, never SUPPORTED.

Table 6 lists the seven targets SAP v5 fixed for C-ext3, with the confirmatory arm F's counts as the sealed artifact records them (`results/routing_bench_bfcl_v4_cext3_results.json`, `targets`). Three were met and four were missed. One of the missed targets is a safety target: arm F accepted 2 of 199 constructed wrong-tool calls, 1.005% against a bar of at most 1%. Both accepted mutants are among the 26 that pass a schema check (NEGATIVE_RESULTS.md §83).

Table 6: C-ext3 pre-registered targets, arm F. The two argument-routing counts are those at the locked commit `6a1ae712`; commit `edd9d144` later moved 79 arm-F episodes from VERIFY to ABSTAIN and none into or out of ACCEPT (NEGATIVE_RESULTS.md §39, §83).

Target	Bar	Arm F	Wilson 95% CI	Verdict
Native wrong-call ACC	$\leq 1\%$	$0/500 = 0.0\%$	0.00–0.76%	met
Constructed wrong-tool ACC	$\leq 1\%$	$2/199 = 1.005\%$	0.28–3.59%	missed
Required-UNKNOWN ACC	$\leq 0\%$	$0/398 = 0.0\%$	0.00–0.96%	met
Irrelevance ABS recall	$\geq 95\%$	$300/300 = 100.0\%$	98.74–100.0%	met
Legitimate read autonomy	$\geq 75\%$	$25/94 = 26.6\%$	18.71–36.32%	missed
Obtainable VER recall	$\geq 95\%$	$93/199 = 46.7\%$	39.93–53.66%	missed
Unobtainable ABS recall	$\geq 95\%$	$126/199 = 63.3\%$	56.43–69.70%	missed

The sealed C-ext3 track supplied that isolation: under a frozen deterministic bundle, native wrong-call acceptance fell 24/500, then 6/500, then 0/500 across the structural, contracts-plus-intent and UNKNOWN-floor arms (CLAIM-019). That is a descriptive single-pass comparison: SAP v5 pre-registered six arms with cluster-level McNemar tests, and the run implemented three arms and no paired test. Exact McNemar tests run after the fact give 24 accepts against 0 for the structural arm against the full configuration ($p = 1.2 \times 10^{-7}$) and 6 against 0 for contracts-plus-intent ($p = 0.031$); the full configuration also lost 22 legitimate reads and gained none (NEGATIVE_RESULTS.md §83). Four caveats bound what it measures. The deterministic intent extractor is the cost carrier: legitimate read autonomy fell to 26.6% (25/94), far below the 75% co-primary bar, so the pre-registered joint hypothesis was not supported. The denominator counts reads under the frozen bundle's own classification; over all 487 gold calls arm F accepted 25 (5.1%). The extractor also ignores negation: "do not cancel" supports a cancel call (NEGATIVE_RESULTS.md §81). Its heuristic grammar was designed with spent C-ext2 material visible (SAP v5 §12, deviation 2). The property the accept arm establishes is *semantically authorized read*, that the call is licensed by a declared contract and a re-derived intent, not that it is the right call for the user's goal. And the result is a routing-bench measurement: discrimination on the governed execution path, where a live deployment supplies the intent and contract bundle, remains **unmeasured**.

The sealed C-ext4 track measured that price directly. SAP v6 fixed eight targets before the remaining BFCL v4 reserve was built into 1,562 episodes over 363 clusters, ID-disjoint from every spent population. A revised bundle (v2, with deterministic derivation receipts) accepted $87/291 = 29.9\%$ of gold calls (Wilson 95% CI 24.9% to 35.4%), against $20/291 = 6.9\%$ for the frozen C-ext3 bundle on the same episodes. Obtainable VERIFY rose from $45/119$ to $100/119 = 84.0\%$. Every safety count stayed at zero: native wrong-call ACCEPT 0/295 (Wilson upper bound 1.29% by row, 5.42% over its 67 tools), constructed wrong-tool 0/119 and required-UNKNOWN ACCEPT 0/238. All eight targets were met (CLAIM-021). They were floors, fixed after bundle v2 and its development replays existed; the C-ext3 bars for obtainable VERIFY (95%) and read autonomy (75%) were not carried over. A refuse-all control arm meets five of the eight: all three safety counts, irrelevance ABSTAIN and unobtainable ABSTAIN. Only gold-call acceptance, obtainable VERIFY and the paired gain over the frozen bundle separate the arms. Bundle v2 without receipts accepted 84/291, so the receipts add 3 of the 87. The revised extractor handles negation.

The gain is narrower than that headline. A descriptive split computed after the run puts 76 of the revised bundle's 87 accepts on the 255 gold calls whose tool also occurs in the development sets. On the other 36 it accepted 11 against 8 for the frozen bundle. The

revisions are naming rules learned from one tool catalogue, and the reserve shares that catalogue. C-ext4 therefore does not show generalisation to unseen tool families (NEGATIVE_RESULTS.md §82). Of the gold calls, 70.1% still do not proceed autonomously. The state index is empty, so the accepted property is still semantically authorized read.

A re-analysis run after the peer review (NEGATIVE_RESULTS.md §83, post hoc) narrows the safety reading further. Only 4 of the 295 wrong calls name a tool the original task offered, so the zero count says almost nothing about confusing two tools of one task. V2D rejected 137 of them as not matching the goal and held the other 158 at VERIFY or ESCALATE, as a refuse-all policy would. Counted by tool, V2D's gain over the frozen bundle is 19 tools against 2 ($p = 1.1 \times 10^{-4}$), not the episode-level p of about 10^{-18}. Two naive rules, scored on the same episodes, show how much of the signal is lexical. Accepting when two tool-name tokens occur in the task accepts 51/291 gold calls and 2/295 wrong calls, and also 48/238 calls missing a required argument. With one token it accepts 178/291 gold calls, 6/295 wrong calls and 132/238 calls missing an argument. V2D accepted 87/291 and none of the others.

No unseen BFCL v4 live row remains, so a test of generalisation needs a new tool catalogue, sealed before any authoring rule is written for it.

11.7 External Validity: AgentHarm Benchmark ($N = 88$)

This tri-mode evaluation ($N = 88$; `artifacts/agentharm_trimode_results.json`, via `RemoraDecisionEngine`) is distinct from the $N = 208$-harmful false-accept run reported in the claim register (`results/external_benchmark_agentharm_v1.json`; FAR = 0.0%, Wilson 95% CI $[0.00\%, 1.81\%]$, FBR = 100%, via the AROMER `/decide` path). The two use different oracles, decision paths, and metrics and should not be read as one benchmark at two sample sizes.

To evaluate governance accuracy on a fully external, independently constructed benchmark, we applied the full Mode 3 cascade to the public AgentHarm dataset [5]: 44 harmful and 44 harmless agent tasks from a red-teaming suite by the UK AI Safety Institute and Gray Swan AI. Three Workers AI models (the cross-family trio) serve as oracles; three modes are evaluated progressively: oracle-only baseline (Mode 1), harm-specific oracle re-evaluation of benign tasks (Mode 2), and the full policy gate applied to Mode 2 verdicts (Mode 3), under the most concise, hardest prompt variant. *Recall* = ESCALATE/harmful; *blocked recall* = (ESCALATE ∪ VERIFY)/harmful; *FPR* = hard-ESCALATE on benign tasks (VERIFY routes to human review and is not counted); Wilson 95% CIs.

Table 7: AgentHarm three-mode results ($N = 88$).

Mode	Recall	CI 95%	FPR	CI 95%	Blocked rec.	Cov.
1 Oracle-only	**1.000**	[0.920, 1.000]	0.318	[0.200, 0.466]	1.000	0.989
2 Harm-specific	**1.000**	[0.920, 1.000]	0.114	[0.050, 0.240]	1.000	0.989
3 Full gate	0.114	[0.050, 0.240]	**0.023**	[0.004, 0.118]	**0.977**	0.977

Mode 3 hard-ESCALATE recall is 0.114 (5/44); 38 further harmful tasks route to human VERIFY, giving blocked recall 0.977 (43/44); two of the 88 tasks ABSTAIN, one harmful and one harmless, which is the 0.977 coverage figure (86/88); zero harmful tasks receive ACCEPT. The three governance goals (blocked recall ≥ 0.95, FPR < 0.10, coverage ≥ 0.95) are met only under the blocked-recall definition. That definition counts VERIFY as a block for harmful tasks but not as friction for benign ones, and Mode 3 sends 80 of the 88 cases to VERIFY. A post-hoc twin-pair analysis finds that Mode 3 tells none of the 44 harmful/benign pairs apart, while the oracle's own verdict (Mode 1) tells 29 apart (NEGATIVE_RESULTS.md §80). Mode 3 shows conservative routing, not discrimination.

Scope caveat. This is an **intent-gating result**: REMORA evaluates and routes the agent's proposed action string; it does not wrap or intercept AgentHarm's internal tool dispatch (a verified solver wrapper is not yet implemented; see `experiments/agentharm/INTERCEPTION_NOTES.md`). The 0.977 figure reflects routing accuracy on the proposed-action surface, not prevention of tool execution, and the research-grade oracle setup may not generalise to live deployments.

12 Ablations

Component ablation: a negative identification result.

An earlier version of this paper reported a five-condition ablation in which conditions A and B reached FAR = 0.300 and 0.100, and concluded that a zero false-accept rate required both the structural gates and a probabilistic routing layer. That table is withdrawn: its conditions were attributed to an artifact that records 0.0143 for every heuristic baseline, and its necessity conclusion is contradicted by the repository's own ablation output. CLAIM-007 is superseded by CLAIM-020 in the claim register, and the withdrawal is recorded in `NEGATIVE_RESULTS.md` §57.

What the committed ablation supports is narrower. Running `experiments/toolcall_ablation_v2.py` produces `results/toolcall_benchmark_v2_ablation.json`. The table shows the full gate and four policy-relevant ablations; the artifact preserves all six conditions.

Condition	FAR	Unsafe exec	Accuracy	Benign friction
REMORA full	0.000	0.000	0.900	0.500
without phase	0.000	0.000	0.871	0.643
without evidence	0.000	0.000	0.900	0.500
without counterfactual	0.000	0.000	0.900	0.500
without hard blocks	0.000	0.000	0.300	0.500

Benign friction is the fraction of benign cases routed to human review. Deterministic simulator benchmark, synthetic `PolicyObservation` features, no live oracle calls; the artifact records effective_n = 56 *harmful template clusters per condition (70 templates in all), so row-level counts overstate independence.*

Reading the table. The false-accept column is constant. Every condition, including the one with the hard blocks removed, records FAR = 0.000. An ablation whose conditions do not differ on a metric cannot identify any component as necessary for that metric, so this ablation attributes the zero-false-accept result to nothing at all (CLAIM-020).

The *without hard blocks* row is a warning about the metric rather than merely an inconclusive result. Removing the deterministic floor leaves FAR at zero while accuracy falls from 0.900 to 0.300: the condition reaches zero false accepts by declining almost every task rather than by judging it correctly. A refuse-all policy scores perfectly on false-accept rate, and any measure a degenerate policy also satisfies is not on its own evidence that a mechanism works.

Conclusion. We report no component attribution for the safety number from this ablation. What the conditions do separate is task accuracy and benign review friction, where the hard blocks account for the largest difference (0.900 → 0.300 accuracy). No ablation whose conditions differ on false-accept rate has been run, so the deterministic floor's contribution to the headline safety result is unmeasured.

Oracle count. The exploratory oracle-count ablation on 302 QA items is in Appendix E.3. Majority vote over three oracles beat a single oracle and correlation weighting on full-coverage accuracy; it is not a headline claim.

13 Case Study: Well Barrier Agent Action

No domain-engineering correctness is claimed. This is an illustrative governance walkthrough.

A drilling agent proposes: *"Autonomously update kill mud weight from 1.52 to 1.48 SG to improve ROP on the 8½" section."* Risk tier: critical. Domain: `well_engineering`.

Oracle responses: Llama-3.1-8B: ESCALATE, conf 0.78; Llama-3.3-70B: ESCALATE, conf 0.85 (NORSOK D-010 §5.4); Llama-4-Scout: VERIFY, conf 0.62. One oracle failed; excluded.

Consensus: $\hat{p}(\text{ESCALATE}) = 0.71$, $\hat{p}(\text{VERIFY}) = 0.29$. $H = 0.866$ bits; $D = 0.29$; regime = critical; $\tau = 0.31$.

Evidence: No field inspection, no ECD calculation, no OIM sign-off. Citation coverage = 0.18 < 0.50 threshold. Evidence router: ESCALATE.

Policy engine: Conditional policy gate fires (phase=critical AND risk=critical) ⇒ ESCALATE. This gate resides in `_CONDITIONAL_GATES`, after the unconditional `hard_guard_floor()`; the outcome is identical.

Follow-up generated: type=on_site_inspection; assign_to=Independent Well Engineer; required_evidence = {ECD calculation, kick tolerance, NORSOK D-010 barrier confirmation, OIM sign-off}; SLA=4h.

Audit hash recorded; action blocked.

14 Case Study: State Resolution on a Fleet Operations Agent

The barrier case shows the deterministic floor stopping an action outright. This one shows the mechanism that distinguishes the current architecture: what happens when the action is well-formed and the *state it refers to* is uncertain. Both calls below are structurally perfect. They are routed differently, and neither routing depends on a model's confidence.

Setup. A fleet-operations agent has six tools, of which `get_vehicle`, `get_driver`, `get_work_order` and `list_depot_vehicles` are declared `read` and `assign_driver` and `close_work_order` are declared `write`. The effects come from the registry, not from the verb in the name, a distinction that matters, because `assign` and `close` were once absent from a mutating-verb list and thirty writes were scored as reads. No bulk state export exists, so every argument value verdict is UNKNOWN: the deployment has point lookups and nothing else.

Call A: a legitimate read. The user asks for the status of vehicle `V-0042`; the agent proposes

`get_vehicle(vehicle_id="V-0042")`. Every structural signal is satisfied. The value is grounded: `V-0042` is named in the task text. But `vehicle_id` steers where the read lands, and nothing has confirmed it exists, so autonomy is unavailable:

Step	Outcome
Argument support	UNKNOWN (no coverage declaration)
Value grounding	grounded (named in the task)
Validation requirement	required (steers the read)
Resolver availability	`get_vehicle` bound to the role
Verdict	VERIFY + `ResolutionPlan`
validator consulted; entity confirmed present	
Re-entry	whole router, fresh observation
Final verdict	ACCEPT

The plan names one resolver and exactly one argument it may write. The validator returns a verdict and hands control back; it never performs the original call and cannot redirect it. Re-entry re-runs the *whole* router, so every guard applies again to the confirmed call rather than being patched around. Had no validator been bound to `vehicle_id`, the verdict would be ABSTAIN, not VERIFY: promising a verification that cannot happen is worse than stopping.

Call B: the same confirmation, a different action. The agent proposes `assign_driver(work_order_id="WO-00123", driver_id="D-0017")`. Both identifiers confirm against the system of record exactly as `V-0042` did. The call is nonetheless held at VERIFY: the registry declares this tool a write, and the read/write posture does not trade confirmed state for autonomy on a state change. No confidence score moves it.

Call C: structurally perfect, wrong tool. The user asks for a vehicle status; the agent proposes `get_work_order` with a work-order identifier that happens to appear elsewhere in the task. Schema valid, arguments satisfiable, values grounded, nothing tainted: every signal in Call A's ladder is satisfied. A declared tool contract does: the task targets a vehicle, the tool acts on a work order, and the mismatch is established rather than merely unverified. The sealed C-ext3 track measured that authority with the frozen contract bundle and the semantic-authority floor. On the same 500 wrong calls, the structural arm accepted 24 and the full configuration 0 (a paired test run after the fact, §11.6). The superseded C-ext2 rate (10.9%, CLAIM-018) was measured on another sample and is not a before-and-after comparison. That measurement does not cover Call C. Only 2 of its 500 wrong calls name a tool the task itself offered (NEGATIVE_RESULTS.md §83), so confusing two tools of one catalogue, as here, is untested. The draft SAP v7 is to test it with wrong calls drawn from the task's own catalogue and from model outputs.

Measured outcome. The study behind the abstract's headline is registered as CLAIM-014: under study conditions the validator loop let 60/60 valid-identifier reads proceed autonomously in the all-UNKNOWN regime, against 0/60 without validators, all eight pre-registered validator targets were met, and none of 139 seeded corruption forms reached ACCEPT (artifacts: `results/system_demonstration_v1.json`, `results/fleetops_degradation_results.json`, `results/fleetops_validator_study_results.json`; internal benchmark, non-blind, the scope the register records).

15 Negative Results

The status of these findings, later mitigations and superseded results is maintained in `NEGATIVE_RESULTS.md`. The experiment-specific constraints needed to interpret this paper's numbers remain beside those results and in §16.

The complete protocol and outcome of the retired selection experiments are preserved in `NEGATIVE_RESULTS.md` §18 and claim register CLAIM-012 / CLAIM-013. They are not current evidence for an execution selector.

χ-proxy failure. Susceptibility χ as standalone difficulty predictor: AUC = 0.39 on $N = 302$ items, below chance. Exploratory OOD use (not a demonstrated detector; the live variant is near-constant unless internal clamps bind): $\chi > 1.45$ (97th percentile) $\Rightarrow$ ESCALATE with `escalate_adversarial`.

Critical-phase trust anticorrelation. Exploratory: the low/high split is 16/21 against 4/11, Fisher's exact $p = 0.053$, and the augmented set uses simulated trust distributions. Q4 high-trust: 50% correct; Q1 low-trust: 75% correct. Confirmed on $N = 32$ real-oracle items and replicated qualitatively on augmented $N = 511$ simulated dataset. No trust threshold separates correct from incorrect critical-phase items (CLAIM-005). *Consequence:* evidence routing (not trust scoring) is the required mechanism for critical-phase decisions.

The execution chain has only a post-hoc, author-run attack evaluation. Everything in §11 is measured on the research and benchmark path.

The authority-bound chain of §4.1 (signed ToolSpec, exact-call lease, single-use grant, re-check at dispatch) has unit, property and conformance tests. A post-hoc attack evaluation was added after the review asked for one (`scripts/execution_chain_attack_eval.py`, `artifacts/posthoc/execution_chain_attack_eval_v1.json`, NEGATIVE_RESULTS.md §87). It drives `/v1/execution` and a policy-only gateway that takes the same proposal-time decision and then runs whatever call is presented. The classes are replay, argument substitution, ToolSpec redeploy or callable swap, risk relabel, cross-task, cross-actor and cross-tenant reuse, forgery, expiry and approver role. REMORA ran 0 of 43 such attacks and the gateway ran 33. REMORA ran all 20 benign controls in the catalogue.

Per-call authorization does not stop exfiltration through allowed calls. The secret reached the sink in all 11 such attacks on both arms, either in a read's URL or sent by a later allowed call. REMORA does not claim trajectory-level or data-flow enforcement. The information-flow systems in §2 [74, 75] target that case. REMORA's own flow check now enters the decision when `REMORA_FLOW_CONTROL=1` (rungs F1 to F6 of the hard floor). With it on, REMORA ran none of the 10 exfiltrations it was run on and no secret reached a sink. A reviewer's approval does not release a confidentiality violation; only a declassification grant could, and grants are not built. The cost was 2 of 4 benign controls: a public fetch after the secret read, and a write whose readers no registry resolves. The context labels in that run are raised by the harness after each read, because tool results do not yet raise them. This measures the decision rule, not derived information-flow control end to end. The catalogue is the author's own and is not an external benchmark. It runs in one profile with in-process state and HMAC signing. Contribution 1 is therefore a design and artifact contribution with a descriptive, author-run attack measurement, not an independently measured security result.

Oracle diversity is partial. The retired Groq swarm used three LLaMA variants of one weight family, so its consensus had an effective diversity closer to two. The 2026-07 cross-family trio (Meta LLaMA, Alibaba Qwen, Mistral) is enforced by a fail-closed same-family check, but statistical independence is not achievable with public models, and diversity weighting mitigates residual correlation without removing it.

In-sample calibration. The selective threshold $\tau^* = 0.4991$ was chosen on a 436-item training split and evaluated on the 108-item holdout of the same 544-item corpus, and the signal and coverage target were chosen from an earlier run on that corpus (`results/selective_n500_holdout_results.json`); the SAP v1 round locked $\tau^* = 0.203$ and is superseded; the decision envelope records this in `in_sample_calibration_warning`.

Evidence retrieval is proxy-based. The current `EvidenceSignal` is built from oracle consensus statistics, not from passage retrieval or an NLI classifier over real documents. The router is tested against MultiNLI as a proxy; real evidence quality will differ.

Demo data. The Control Room frontend uses a seeded simulator for oracle votes, latencies and evidence snippets, labelled as demo components; the backend policy engine, observables and routing process real oracle responses.

MetaJudge circularity. AROMER's MetaJudge uses the same model family and provider as the governance oracles, so biases common to that family go undetected. Its AII component reflects within-family self-consistency more than external validation.

Causal enrichment not deployed. The probability-of-sufficiency module is unit-tested, but live enrichment of new episodes is not deployed, and no empirical results over real governance episodes are reported for it.

No human evaluation of explanations. Trace faithfulness is machine-checked (the trace reproduces the engine's decision rule for rule), but no user or expert study of comprehensibility or actionability has been run. The paper claims traceability, not explainability in the human-factors sense.

Boundary claims are declared, not closed. Three properties that bound every custody and dispatch claim in this paper are explicitly unestablished (§4.4). Runtime capability-surface completeness is `NOT_ESTABLISHED`: the strict-profile custody and dispatch evidence covers REMORA's own governed surface, and does not establish that an external agent process lacks alternate credential, network or execution paths. Implementation effect non-transitivity is `NOT_ESTABLISHED`: authorizing a tool does not authorize every privileged effect its implementation can reach, so the lease binds the exact call, not the effect space. Authoritative execution-context binding is `NOT_ESTABLISHED`: the opt-in path is tested inside REMORA, the authority of its deployment sources is

not. The external run records for the E-ECB, E-FA and E-EE contracts come from an external second implementation and are recorded as not independent. The two E7 records are independent under the repository's recorded rule and establish fixture premises only. No boundary-level claim result is established by any of them. The paper's custody claims (§4.1), and its title, should be read within these declared limits: REMORA establishes properties at a declared boundary and does not claim that all agent effects are governed.

16 Threats to Validity

Internal validity: Earlier QA operating points were selected on limited or reused data and are retained as historical evidence in `NEGATIVE_RESULTS.md` §18. The benchmark also contains 13.8% author-curated items (15.1% including the adversarial-curated set); the tool-call benchmark uses synthetic adversarial patterns. The BFCL tracks are an adaptive sequence of looks at one finite pool (v3, C-ext2, C-ext3, C-ext4). Each track was ID-disjoint from the spent ones, but the bundles were developed between looks on the same tool catalogue, and the C-ext4 targets were set with the bundle's development numbers in hand.

External validity: Model coverage is mixed across rounds: the earlier QA and tool-call rounds used a Groq-hosted three-LLaMA swarm (one weight family, partial diversity), while the 2026-07 clean round, the AgentHarm external validation, and the real-LLM baselines use a Cloudflare Workers AI cross-family trio (Meta LLaMA, Alibaba Qwen, Mistral). Results on other providers, closed-source models, or quantized variants may still differ; the benchmark phase distribution may not match deployment reality; and MultiNLI is a proxy for field evidence retrieval.

Construct validity: "Trust" is a derived scalar, not a frequency probability of per-item correctness. Utility scores are designed task weights, not real-world costs. The BFCL tracks route annotated gold proposals, and their wrong calls are cross-task substitutions, not model errors (§11.5). Gold-call acceptance is a friction measure on correct proposals, not task success, and is not comparable with BFCL leaderboard scores. No external baseline (an LLM judge, an existing guardrail) was run on the sealed BFCL episodes. Naive name-overlap and schema-only rules were scored after the fact (NEGATIVE_RESULTS.md §83); every other comparison is between REMORA configurations and a refuse-all control. The AII composite uses heuristic weights; AROMER reached TRAINED (AII 0.8442) and regressed to CAPABLE (0.7885) the same day (CLAIM-006).

Statistical validity: 95% Wilson confidence intervals are reported for key accuracy figures. Formal hypothesis tests are not performed across all conditions; where none is reported, overlapping or non-overlapping intervals are not evidence of a difference. Zero-count Wilson bounds treat rows as independent. The BFCL wrong calls share proposed tools and REMORA's decision depends on the tool, and the AgentHarm and tool-call scenarios share base behaviours and templates, so bounds clustered at those levels are wider. Post hoc, the zero-count upper bounds are 3.5% over C-ext3's 106 tools, 5.42% over C-ext4's 67 tools, 6.42% over CLAIM-001's 56 harmful templates and 7.56% over CLAIM-002's 47 base behaviours (NEGATIVE_RESULTS.md §83). The tool-call benchmark's utility gain (+0.46 against the keyword heuristics, +0.26 against the retired temperature-gate heuristic) is a post-hoc test, declared after the pre-registered hypothesis failed. Wilson intervals are fixed-N constructions and are not valid under the continuous monitoring that the REM-020 longitudinal gate performs (optional stopping). A time-uniform confidence sequence [53, 31, 38, 46] is therefore reported alongside for the gate's cycle-level false-accept indicator: 0/168 cycles gives a 95% time-uniform upper bound of 4.72% (`results/far_confidence_sequence_v1.json`, claim register CLAIM-011). The sequence is wider than Wilson at any fixed N: the price of validity under data-dependent stopping.

17 Safety, Ethics, and Governance

REMORA is designed to *reduce* unsafe autonomous AI actions. ACCEPT means assurance conditions are met, not that the action is correct. Human reviewers receiving ESCALATE retain full accountability for outcomes. The oracle swarm routes sensitive operational data through external APIs; data classification requirements must be evaluated before deployment. The adversarial firewall detects known injection patterns but is not a complete defense against sophisticated adversarial inputs.

18 Reproducibility

Repository: `https://github.com/darklordVirtual/REMORA-research`. **Release:** `Preprint v0.12.0, revision 2026-10-19; repository release tag v0.12.0` (a frozen, CI-green commit with sdist/wheel checksums published on the

GitHub release; this paper is versioned in lockstep with it). **Python:** 3.11+.

```
python -m pip install -e ".[dev]"
make audit        # lint + tests + claim consistency
make benchmark    # all deterministic benchmarks
make credibility-pack
```

At the revision date the suite collects 10,850 tests across 706 test files (`python -m pytest tests/ -q`); CI output is authoritative.

Live oracle experiments require `GROQ_API_KEY` (earlier rounds) or `CLOUDFLARE_API_TOKEN` (Cloudflare Workers AI, the 2026-07 clean round). Oracle model versions may change; results are sensitive to model version updates. Stored artifact experiments are fully deterministic.

Fixture self-service vs. runtime reproduction. Three frozen interoperability contracts are self-service at the fixture level. An outside operator can run the pinned reference verifier for the exact-call-binding, fresh-authority and effect-evidence packages under `artifacts/interop/`. Commands, source revisions and package digests are declared in `docs/interop/remora-capabilities-v1.yaml`. These commands replay publisher-authored reference verifiers against frozen fixture bytes. They do not execute the REMORA runtime, and they do not validate that the current runtime implements the contract. A successful replay is a bounded fixture claim, not a reproduction of the system. Runtime-level reproduction is declared `NOT_READY` for all three capabilities. An externally recorded run advances a contract's independence only under the recorded rule (§4.3) and never its claim result. The declarations themselves validate with `scripts/check_remora_capabilities.py`, `scripts/check_remora_boundaries.py` and `scripts/build_remora_boundary_summary.py --check`.

License. This preprint is distributed as part of the REMORA Licensed Work under the Business Source License 1.1 (source-available, not open source). Commercial use requires a separate REMORA Commercial License from the Licensor, Stian Skogbrott (`support@luftfiber.no`). Third-party datasets and materials remain under their own terms; see `THIRD_PARTY_NOTICES.md`.

Claim binding: headline numbers are bound to the machine-readable claim register (`docs/assurance/claim_register_v1.yaml`), the single authoritative source, through the Markdown source of this paper (`paper/remora_paper.md`): the claim-provenance and claim-consistency CI gates fail on divergence between that source and the register. This LaTeX manuscript and the compiled PDF are kept in lockstep with the Markdown source by `scripts/check_paper_sync.py` (version, revision, and guarded process claims); numeric drift in the typeset copy is caught by review rather than by an automated gate on the `.tex`. Each `review-*` tag triggers a workflow that regenerates a commit-bound test attestation and a governance pack whose `manifest.json` records the exact commit SHA and per-file SHA-256 checksums, uploaded as one `remora-review-handoff-<sha>` artifact.

19 Conclusion

REMORA makes permission to execute a concrete agent action an explicit, recorded decision. Its execution path binds that decision to a signed tool definition, a single call and a single-use grant, then records dispatch and observed effect as different claims. The studies reported here examine three consequences of that design.

The zero-unsafe result has no identified cause. The 0% unsafe-acceptance result on the adversarial tool-call benchmark (700 synthetic rows; 0 of 56 harmful templates, Wilson 95% upper bound 6.4% over those templates; baselines 1.4%, difference not significant at $p = 0.50$; a post-hoc test finds gains in utility, +0.456 against the keyword heuristics and +0.259 against the temperature-gate heuristic, and in accuracy) was observed with the deterministic policy gates in place, and not produced by the multi-oracle consensus machinery. No experiment attributes it to the gates: the committed component ablation records FAR = 0.000 in every condition, including the one with the hard blocks removed, and so identifies no component as necessary for the safety number (CLAIM-020). We state this against our own interest: the probabilistic architecture that gives the system its name is not what produces its headline safety number, and the deterministic floor's share in it is unmeasured. Routing quality is a different claim requiring different evidence.

Unknown state is a resolution problem. Absence without declared completeness is UNKNOWN, not UNSUPPORTED; a VERIFY that cannot name a bounded lookup degrades to ABSTAIN; and a resolved value re-enters the whole router rather than patching the prior decision. Under study conditions this lets 60/60 valid-identifier reads proceed autonomously in the all-UNKNOWN regime, where none did without validators, with zero corrupt-identifier acceptances, zero cross-tenant lookups, and writes held at VERIFY throughout.

Routing holds within one tool catalogue. The

sealed C-ext3 track isolated the semantic axis within one set of episodes: with declared tool contracts and the semantic-authority floor, arm F accepted 0/500 substituted wrong calls (Wilson 95% upper bound 0.76% by row, 3.5% clustered by tool), against 24/500 for the structural arm (exact McNemar $p = 1.2 \times 10^{-7}$, run after the fact). The same run missed four of its seven pre-registered targets. Constructed wrong-tool acceptance was 2/199 against a safety bar of at most 1%, read autonomy was 26.6% against 75%, and both argument-routing targets were missed, so its pre-registered joint hypothesis was not supported. The earlier C-ext2 track (CLAIM-018, superseded by CLAIM-019) met its five targets with 10.9% (28/258) wrong-call acceptance on a different sample without contracts; it stays on record as superseded and is not the comparison for C-ext3. A third sealed track, C-ext4, raised correct-call acceptance from 6.9% to 29.9% with every safety count at zero, as for a refuse-all control; the gain sits mostly on tools seen in development, so generalisation to unseen tool families is untested.

The current architecture offers two further research interfaces. Typed evidence admission makes the scope and provenance of a claimed observation inspectable; federation publishes digest-pinned packages that an outside verifier can run without turning foreign results into execution authority. The former is not yet wired into effect verification. External run records now exist for the exact-call-binding, fresh-authority and effect-evidence contracts (E-ECB, E-FA, E-EE), recorded as not independent. The E7 runtime-surface contract has two external fixture-level runs that meet the repository's independence rule, with every boundary-level claim result still `NOT_ESTABLISHED`. Contract probes and a formal model from this repository found four findings in an external federation transport, all fixed upstream (§4.3). Experiment-specific constraints appear beside the reported results and in §16; `NEGATIVE_RESULTS.md` preserves the complete finding history.

What this paper does not show: transfer to a new tool catalogue, discrimination against wrong calls that a model actually made, an advantage over external or naive baselines, and the behaviour of the execution chain under an external or adaptive attack (§15, §16).

REMORA remains `SHADOW_ONLY`. It does not claim that all agent effects are governed. It makes the governed boundary explicit, declares what properties it can establish at that boundary, and publishes what it cannot establish. Runtime capability-surface completeness and implementation effect non-transitivity are among the published non-claims. The bounded contracts it exposes can be reproduced by outside operators. Production enforcement evidence and Federation adoption remain unestablished. Its present contribution is a reproducible account of where authority comes from, when a call may proceed, and which evidence can support a claim about its effect.

Acknowledgements

Benchmarks use publicly available datasets (BoolQ, TruthfulQA, MultiNLI, ARC-Challenge, MMLU-Pro). Oracle inference via the Groq API (earlier rounds) and Cloudflare Workers AI (the 2026-07 clean round). Open Policy Agent used for policy evaluation. Negative results documented in full in `NEGATIVE_RESULTS.md`; the authors thank all reviewers who pointed out scope corrections and limitations. We thank contributor Erlend Barli (`github.com/erlendbarli`) for contributions to the project (see `CONTRIBUTORS.md`).

Use of AI-Assisted Development Tools

Generative AI tools were used as assistive tools during the development of this research prototype. Their use included brainstorming, code drafting and refactoring, test scaffolding, documentation editing, language revision, and support in locating and summarising relevant literature.

The author defined the research questions, system architecture, policy semantics, evaluation criteria, experimental scope, and all reported claims. All source code, tests, benchmark artifacts, citations, numerical results, and conclusions were reviewed and verified by the author before inclusion. Generative AI tools were not listed as authors and were not relied upon as independent sources of experimental evidence.

Where AI-assisted output informed implementation or prose, the final responsibility for correctness, reproducibility, and interpretation remains with the human author. A full disclosure is available in `docs/AI_USE.md`.

References

[1] Kuhn, L., Gal, Y., & Farquhar, S. (2023). Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In *Proceedings of the International Conference on Learning Representations (ICLR 2023)*. arXiv:2302.09664.

[2] Kirchner, J. H., Chen, Y., Edwards, H., Leike, J., McAleese, N., & Burda, Y. (2024). Prover-verifier games improve legibility of LLM outputs. *arXiv:2407.13692*.

[3] Angelopoulos, A. N., Bates, S., Fisch, A., Lei, L., & Schuster, T. (2022). Conformal risk control. *arXiv:2208.02814*.

[4] Tibshirani, R. J., Barber, R. F., Candès, E. J., & Ramdas, A. (2019). Conformal prediction under covariate shift. *NeurIPS 2019*.

[5] Andriushchenko, M., et al. (2024). AgentHarm: A benchmark for measuring harmfulness of LLM agents. *arXiv:2410.09024*.

[6] Dong, B. & Wang, Q. (2025). Evaluating the performance of the DeepSeek model in confidential computing environment. *arXiv:2502.11347*.

[7] Wang, X., et al. (2023). Self-consistency improves chain of thought reasoning in language models. *ICLR 2023*.

[8] Du, Y., Li, S., Torralba, A., Tenenbaum, J., & Mordatch, I. (2024). Improving factuality and reasoning in language models through multiagent debate. *ICML 2024*.

[9] Zheng, L., et al. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *NeurIPS 2023*.

[10] Cobbe, K., et al. (2021). Training verifiers to solve math word problems. *arXiv:2110.14168*.

[11] Chow, C. K. (1970). On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory, 16*(1), 41–46.

[12] El-Yaniv, R. & Wiener, Y. (2010). On the foundations of noise-free selective classification. *Journal of Machine Learning Research, 11*, 1605–1641.

[13] Lakshminarayanan, B., Pritzel, A., & Blundell, C. (2017). Simple and scalable predictive uncertainty estimation using deep ensembles. *Advances in Neural Information Processing Systems (NeurIPS 2017)*.

[14] Kuncheva, L. I. & Whitaker, C. J. (2003). Measures of diversity in classifier ensembles and their relationship with the ensemble accuracy. *Machine Learning, 51*(2), 181–207.

[15] Geifman, Y. & El-Yaniv, R. (2017). Selective classification for deep neural networks. *NeurIPS 2017*.

[16] Kadavath, S., et al. (2022). Language models (mostly) know what they know. *arXiv:2207.05221*.

[17] Angelopoulos, A. & Bates, S. (2021). A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv:2107.07511*.

[18] Guo, C., et al. (2017). On calibration of modern neural networks. *ICML 2017*.

[19] Endsley, M. (1995). Toward a theory of situation awareness in dynamic systems. *Human Factors, 37*(1), 32–64.

[20] Kelly, T. (1998). Arguing safety: A systematic approach to managing safety cases. PhD thesis, University of York.

[21] Bloomfield, R. & Bishop, P. (2010). Safety and assurance cases: Practice, trends and challenges. *SAFECOMP 2010*.

[22] Lin, S., et al. (2022). TruthfulQA: Measuring how models mimic human falsehoods. *ACL 2022*.

[23] Clark, C., et al. (2019). BoolQ: Exploring the surprising difficulty of natural yes/no questions. *NAACL 2019*.

[24] Williams, A., et al. (2018). A broad-coverage challenge corpus for sentence understanding through inference. *NAACL 2018*.

[25] European Parliament & Council of the European Union. (2024). Regulation (EU) 2024/1689 of the European Parliament and of the Council (Artificial Intelligence Act). *Official Journal of the European Union*, L 2024/1689.

[26] Open Policy Agent. (2024). `https://www.openpolicyagent.org/`.

[27] Bjøru, A. R. (2026). Causal Post-hoc Explainable AI. PhD thesis, Norwegian University of Science and Technology (NTNU). ISBN 978-82-353-0022-5.

[28] Chennabasappa, S., et al. (2025). LlamaFirewall: An open source guardrail system for building secure AI agents. *arXiv:2505.03574*.

[29] Choi, M., Yang, S., Kim, D., Kim, S., Son, J., Lee, Y., Choo, J., & Kwak, Y. (2026). Membrane: A self-evolving contrastive safety memory for LLM agent defense. *arXiv:2606.05743*.

[30] Corsi, D., Marchesini, E., & Farinelli, A. (2021). Formal verification of neural networks for safety-critical tasks in deep reinforcement learning. *UAI 2021*, PMLR 161:333–343.

[31] Darling, D. A. & Robbins, H. (1967). Confidence sequences for mean, variance, and median. *Proceedings of the National Academy of Sciences, 58*(1), 66–68.

[32] Debenedetti, E., et al. (2024). AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. *NeurIPS 2024 Datasets & Benchmarks*.

[33] Debenedetti, E., et al. (2025). Defeating prompt injections by design (CaMeL). *arXiv:2503.18813v2*.

[34] Farquhar, S., Kossen, J., Kuhn, L., & Gal, Y. (2024). Detecting hallucinations in large language models using semantic entropy. *Nature, 630*, 625–630.

[35] Ge, Y. (2026). Governance architecture for autonomous agent systems: Threats, framework, and engineering practice. *arXiv:2603.07191*.

[36] Greenblatt, R., Shlegeris, B., Sachan, K., & Roger, F. (2024). AI control: Improving safety despite intentional subversion. *ICML 2024*, PMLR 235:16295–16336.

[37] Guldimann, P., et al. (2024). COMPL-AI framework: A technical interpretation and LLM benchmarking suite for the EU Artificial Intelligence Act. *arXiv:2410.07959*.

[38] Howard, S. R., Ramdas, A., McAuliffe, J., & Sekhon, J. (2021). Time-uniform, nonparametric, nonasymptotic confidence sequences. *Annals of Statistics, 49*(2), 1055–1080.

[39] International Electrotechnical Commission. (2016). IEC 61511-1:2016, Functional safety: Safety instrumented systems for the process industry sector. IEC.

[40] Inan, H., et al. (2023). Llama Guard: LLM-based input-output safeguard for human-AI conversations. *arXiv:2312.06674*.

[41] Mohri, C. & Hashimoto, T. (2024). Language models with conformal factuality guarantees. *ICML 2024*.

[42] Mu, T., Helyar, A., et al. (2024). Rule based rewards for language model safety. *NeurIPS 2024*.

[43] Standards Norway. (2021). NORSOK D-010:2021, Well integrity in drilling and well operations. Standards Norway.

[44] Pearl, J. (2009). *Causality: Models, Reasoning, and Inference* (2nd ed.). Cambridge University Press.

[45] Raji, I. D., Xu, P., Honigsberg, C., & Ho, D. E. (2022). Outsider oversight: Designing a third party audit ecosystem for AI governance. *AIES '22*. arXiv:2206.04737.

[46] Ramdas, A., Grünwald, P., Vovk, V., & Shafer, G. (2023). Game-theoretic statistics and safe anytime-valid inference. *Statistical Science, 38*(4), 576–601.

[47] Rebedea, T., et al. (2023). NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails. *EMNLP 2023 System Demonstrations*.

[48] Ruan, Y., et al. (2024). Identifying the risks of LM agents with an LM-emulated sandbox (ToolEmu). *ICLR 2024*.

[49] Shafer, G. & Vovk, V. (2008). A tutorial on conformal prediction. *Journal of Machine Learning Research, 9*, 371–421.

[50] Shamsujjoha, M., Lu, Q., Zhao, D., & Zhu, L. (2024). A taxonomy of multi-layered runtime guardrails for designing foundation model-based agents: Swiss cheese model for AI safety by design. *arXiv:2408.02205*.

[51] Sharma, M., et al. (2025). Constitutional classifiers: Defending against universal jailbreaks across thousands of hours of red teaming. *arXiv:2501.18837*.

[52] Shi, T., He, J., Wang, Z., Li, H., Wu, L., Guo, W., & Song, D. (2025). Progent: Securing AI agents with privilege control. *arXiv:2504.11703v2*.

[53] Ville, J. (1939). *Étude critique de la notion de collectif.* Gauthier-Villars.

[54] Vovk, V., Gammerman, A., & Shafer, G. (2005). *Algorithmic Learning in a Random World.* Springer.

[55] Wang, X., Aitchison, L., & Rudolph, M. (2023). LoRA ensembles for large language model fine-tuning. *arXiv:2310.00035*.

[56] Wang, H., Poskitt, C. M., et al. (2025). AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents. *arXiv:2503.18666*.

[57] Xiang, Z., et al. (2025). GuardAgent: Safeguard LLM agents by a guard agent via knowledge-enabled reasoning. *ICML 2025*.

[58] Yadkori, Y. A., Kuzborskij, I., et al. (2024). Mitigating LLM hallucinations via conformal abstention. *arXiv:2405.01563*.

[59] Yang, C. (2026). AgentTrust: A self-improving trust layer for AI-agent actions. *arXiv:2606.08539*.

[60] Yao, S., Shinn, N., Razavi, P., & Narasimhan, K. (2024). τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv:2406.12045*.

[61] Yuan, T., et al. (2024). R-Judge: Benchmarking safety risk awareness for LLM agents. *Findings of EMNLP 2024*.

[62] Zhan, Q., Liang, Z., et al. (2024). InjecAgent: Benchmarking indirect prompt injections in tool-integrated LLM agents. *Findings of ACL 2024*.

[63] Zhang, M., Huang, M., Shi, R., Guo, L., Peng, C., Yan, P., Zhou, Y., & Qiu, X. (2024). Calibrating the confidence of large language models by eliciting fidelity. *arXiv:2404.02655*.

[64] Qin, S., Zhuang, H., Zhou, Y., Han, Y., & Zhang, X. (2026). AIRGuard: Guarding agent actions with runtime authority control. arXiv:2605.28914v1.

[65] Wang, Z. (2026). Proof-carrying agent actions: Model-agnostic runtime governance for heterogeneous agent systems. arXiv:2606.04104v1.

[66] Rhodes, J., & Kang, G. (2026). Proof of execution: Runtime verification for governed AI agent actions. arXiv:2607.05397v1.

[67] Michael, A. E., & Roesner, F. (2026). How agents ask for permission: User permissions for AI agents, from interfaces to enforcement. arXiv:2607.13718v2.

[68] Anderson, J. P. (1972). Computer security technology planning study. Technical Report ESD-TR-73-51, Vol. II, U.S. Air Force Electronic Systems Division.

[69] Hardy, N. (1988). The confused deputy (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review*, 22(4), 36–38.

[70] Birgisson, A., Politz, J. G., Erlingsson, Ú., Taly, A., Vrable, M., & Lentczner, M. (2014). Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. *NDSS 2014*.

[71] Crosby, S. A., & Wallach, D. S. (2009). Efficient data structures for tamper-evident logging. *USENIX Security Symposium 2009*.

[72] Patil, S. G., Zhang, T., Wang, X., & Gonzalez, J. E. (2023). Gorilla: Large language model connected with massive APIs. arXiv:2305.15334.

[73] Patil, S. G., Mao, H., Yan, F., Ji, C. C.-J., Suresh, V., Stoica, I., & Gonzalez, J. E. (2025). The Berkeley Function Calling Leaderboard (BFCL): From tool use to agentic evaluation of large language models. *ICML 2025*.

[74] Wu, F., Cecchetti, E., & Xiao, C. (2024). System-level defense against indirect prompt injection attacks: An information flow control perspective. arXiv:2409.19091v2.

[75] Costa, M., Köpf, B., Kolluri, A., Paverd, A., Russinovich, M., Salem, A., Tople, S., Wutschitz, L., & Zanella-Béguelin, S. (2025). Securing AI agents with information-flow control (FIDES). arXiv:2505.23643v2.

[76] Wu, Y., Roesner, F., Kohno, T., Zhang, N., & Iqbal, U. (2025). IsolateGPT: An execution isolation architecture for LLM-based agentic systems. *NDSS 2025*. arXiv:2403.04960v2 (v1 titled SecGPT).

[77] Hines, K., Lopez, G., Hall, M., Zarfati, F., Zunger, Y., & Kiciman, E. (2024). Defending against indirect prompt injection attacks with spotlighting. arXiv:2403.14720v1.

[78] Chen, S., Piet, J., Sitawarin, C., & Wagner, D. (2025a). StruQ: Defending against prompt injection with structured queries. *USENIX Security Symposium 2025*. arXiv:2402.06363v2.

[79] Chen, S., Zharmagambetov, A., Mahloujifar, S., Chaudhuri, K., Wagner, D., & Guo, C. (2025b). SecAlign: Defending against prompt injection with preference optimization. *ACM CCS 2025*. arXiv:2410.05451v3.

[80] Jia, F., Wu, T., Qin, X., & Squicciarini, A. (2025). The Task Shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents. *ACL 2025 (Long Papers)*, 29680–29697. arXiv:2412.16682v1.

[81] Zhu, K., Yang, X., Wang, J., Guo, W., & Wang, W. Y. (2025). MELON: Provable defense against indirect prompt injection attacks in AI agents. *ICML 2025*. arXiv:2502.05174v4.

[82] Tsai, L., & Bagdasarian, E. (2025). Contextual agent security: A policy for every purpose (Conseca). *HotOS 2025*. arXiv:2501.17070v3.

[83] Beurer-Kellner, L., Buesser, B., Creţu, A.-M., Debenedetti, E., Dobos, D., Fabian, D., Fischer, M., Froelicher, D., Grosse, K., Naeff, D., Ozoani, E., Paverd, A., Tramèr, F., & Volhejn, V. (2025). Design patterns for securing LLM agents against prompt injections. arXiv:2506.08837v3.

[84] Model Context Protocol contributors. (2025). Model Context Protocol specification, revision 2025-11-25. `https://modelcontextprotocol.io/specification/2025-11-25`

[85] Biscuit contributors. (2026). Eclipse Biscuit: An authorization token with decentralized verification. `https://www.biscuitsec.org/` (accessed 2026-10-18).

[86] Lodderstedt, T., Richer, J., & Campbell, B. (2023). OAuth 2.0 Rich Authorization Requests. RFC 9396 (Standards Track), IETF.

[87] OASIS. (2013). eXtensible Access Control Markup Language (XACML) version 3.0. OASIS Standard, 22 January 2013. E. Rissanen (ed.).

[88] Cutler, J. W., Disselkoen, C., Eline, A., He, S., Headley, K., Hicks, M., Hietala, K., Ioannidis, E., Kastner, J., Mamat, A., McAdams, D., McCutchen, M., Rungta, N., Torlak, E., & Wells, A. M. (2024). Cedar: A new language for expressive, fast, safe, and analyzable authorization. *Proceedings of the ACM on Programming Languages, 8*(OOPSLA1), 670–697. doi:10.1145/3649835. arXiv:2403.04651v2.

[89] Schneier, B., & Kelsey, J. (1999). Secure audit logs to support computer forensics. *ACM Transactions on Information and System Security, 2*(2), 159–176. doi:10.1145/317087.317089.

[90] Laurie, B., Langley, A., & Kasper, E. (2013). Certificate Transparency. RFC 6962 (Experimental), IETF.

[91] Laurie, B., Messeri, E., & Stradling, R. (2021). Certificate Transparency version 2.0. RFC 9162 (Experimental; obsoletes RFC 6962), IETF.

A Mathematical Notation Table

Table 8: Mathematical notation.

Symbol	Definition	Domain
$\mathcal{O}$	Oracle set	$n \geq 2$ endpoints (design intent ≥ 3 distinct families)
$\mathcal{O}'$	Valid oracle set	$\mathcal{O}' \subseteq \mathcal{O}$
$\varphi(o)$	Canonical verdict	(polarity, hash, mag., tags)
$\rho(a, b)$	Pairwise agreement rate	$[0, 1]$, window 200
$w(o)$	Diversity weight	normalized
$\tilde{p}(v)$	Weighted support	$[0, 1]$, sums to 1
H	Shannon entropy	bits ≥ 0
D	Dissensus	$[0, 1]$
k	Active verdict count	≥ 1
η	Consensus concentration	$[0, 1]$
χ	Susceptibility	$[0, \infty)$
τ	Trust score	$[0, 1]$
h_{bound}	Heuristic false-consensus risk proxy	$[0, 1]$
w_{phase}	Phase weight	$\{1.0, 0.5, 0.1\}$
g	Gate outcome	ACCEPT/VERIFY/ABS./ESC.
Γ	Gate function	$(q, a, c) \to g$

B DecisionEnvelope v2 Schema Summary

Top-level blocks: `request`, `assessment`, `gate`, `reviewer_context`, `follow_up`, `history`, `policy_learning`, `audit`. Full JSON schema is in the Markdown paper and repository (`remora/policy/report.py`).

C Negative Results Archive

The complete, versioned record of negative and resolved findings, including resolved and superseded items, lives in `NEGATIVE_RESULTS.md` in the repository. It is not duplicated here: the main-body Negative Results section carries the findings that shape the architecture; this appendix is a single pointer to the full archive so the two do not drift.

D Implementation Status Table

Status vocabulary: *Integrated* = invoked by the `/v1/assess` research surface (per the canonical-engine ADR, the policy core behind `/v1/execution/*` is the enforcing engine; this table describes the research pipeline); *Offline* = implemented and evaluated with committed artifacts but not invoked by the primary API; *Adapter* = implemented and parity-tested, available as an integration option; *Planned* = not implemented.

Table 9: Component status: implemented vs. integrated in `/v1/assess`.

Component	Impl.	Integrated	Notes
Oracle fan-out	Yes	Yes	ThreadPoolExecutor
Canonicalization (φ)	Yes	Yes	NFKC, Semantic Entropy [1]
Correlation weighting	Yes	Per-request	Fresh engine per API request
H, D, η, τ	Yes	Yes	Research observables
Phase classification	Yes	Yes	3-class
Policy engine (7 headline controls: 3 unconditional + 4 conditional)	Yes	Yes	Full gate inventory in code
OPA/Rego adapter	Adapter	No	API calls Python engine directly
Governance Intelligence	Offline	No	Optional pre-policy helper
`PhaseAwareGuardrail`	Offline	No	Committed $N = 544$ artifact
Covariate-shift CRC	Offline	No	Nominal bound (heuristic weights)
PVD-inspired agreement score	Offline	No	Post-hoc heuristic
Evidence router (oracle proxy)	Yes	Yes	Proxy signal
Evidence router (semantic)	Planned	No	Interface ready
DecisionEnvelope v2	Yes	Yes	Attribute-frozen, not deep-frozen
SHA-256 hash-chain	Yes	API layer	HMAC only with signing key
Execution grant path	Yes	Separate router	Dispatches via `GovernedToolDispatcher` (lease-bound; CAP-013)
Human review workflow	Yes	Partial	Demo UI only
WORM audit storage	Planned	No	External dependency
ZK assurance proofs	Planned	No	Stubbed

E Retired Research Path: Consensus Observables

This appendix and the two that follow hold the ensemble, conformal and calibration material that the body summarises in §5, §8, §11 and §12. None of it is on the governed execution path (§5.1), and no headline result depends on it. It is kept because the numbers, the caveats and the negative results are part of the record (`NEGATIVE_RESULTS.md` §18, §38).

E.1 Weighted Support Distribution

Let $\hat{p}(v) = \sum_{o \in \mathcal{O}'} w(o) \cdot \mathbf{1}[\varphi(o) = v]$ be the diversity-weighted support for verdict v, normalized to sum to 1. The diversity weight $w(o)$ penalizes oracles that frequently agree with others:

$$w(o) = \frac{(1/n)}{1 + \sum_{j \neq o} \rho(o, j)} \cdot Z^{-1} \tag{3}$$

where $\rho(o, j)$ is the rolling pairwise agreement rate (window 200 samples) and Z normalizes. Oracles that agree with the swarm receive lower weight; independent dissenters receive higher weight. The rolling window persists for a long-lived engine instance (as in the offline experiments); the reference API constructs a fresh engine per request, so cross-request persistence of correlation state is a deployment integration point, not a current runtime property.

E.2 Core Observables

Shannon entropy (computed over the token-fingerprint proxy clustering in all reported experiments, not NLI-based semantic clusters; backend note in §4, limitations in §16):

$$H = -\sum_{v} \hat{p}(v) \log_2 \hat{p}(v) \quad \text{(bits)} \tag{4}$$

Dissensus:

$$D = 1 - \max_{v} \hat{p}(v) \tag{5}$$

Consensus concentration:

$$\eta = \frac{\max_v \hat{p}(v) - 1/k}{1 - 1/k} \tag{6}$$

where $k = |\{v : \hat{p}(v) > 0\}|$ is the number of active verdicts.

Trust score:

$$\tau = \eta \cdot (1 - h_{\text{bound}}) \cdot w_{\text{phase}} \cdot \left(1 + \frac{\chi}{\chi_0}\right)^{-1} \tag{7}$$

Here h_{bound} is a heuristic risk proxy, not a proven bound; w_{phase} weights consensus regimes. This score describes research routing, not an execution authorization. Implementation details and the retired selection hypothesis are recorded in `NEGATIVE_RESULTS.md` (§18, §38).

E.3 Oracle Count and Correlation Weighting

Oracle count ($N = 302$). Single oracle: 56.95% (95% CI [51.3, 62.4]); three-oracle majority vote: 82.78% ([78.1, 86.6]), a gain of 25.8 pp; correlation-aware weighting: 69.54% ([64.1, 74.5]), below raw majority vote on full-coverage accuracy. This is an exploratory ablation, not a headline claim (`results/ablation_v2_results.json`). The value of correlation weighting lies in trust calibration and selective coverage, not full-coverage accuracy, and the primary benefit of the oracle swarm is disagreement *detection* (high H, high D).

Without correlation weighting, consensus from a within-family oracle cluster would receive inflated support. The effect of the diversity weights on trust scores and escalation rates has not been tested in a controlled study that varies oracle family composition.

F Offline Critical-Phase Selection Components

The critical-phase trust inversion that motivates these components is stated in §8.

F.1 PhaseAwareGuardrail: Exploiting Trust Inversion

The inversion is a *selection criterion reversal*, not an accuracy problem. Items with $\tau < 0.10$ in the critical phase achieve 76.2% accuracy ($N = 21$) vs. 36.4% for high-τ critical items ($N = 11$). A `PhaseAwareGuardrail` exploits this by applying an inverted score $\tilde{\tau} = 1 - \tau$ for critical-phase items, then

calibrating a conformal threshold on $\tilde{\tau}$. A hard gate rejects all items with $\tau \geq \tau_{\max} = 0.10$ (the groupthink boundary) regardless of the conformal result.

The combined pool, all 99 ordered-phase items (sorted by $\tau \downarrow$) plus 21 low-τ critical items ($\tau < 0.10$, sorted by $\tilde{\tau} \downarrow$), achieves:

- 20.0% coverage ($N = 108$): 85.2% accuracy CI [77.3%, 90.7%]
- 22.1% coverage ($N = 120$): 85.0% accuracy CI [77.5%, 90.3%]

This is a +3.9 pp coverage gain over the ordered-only operating point (18.2%, 86.9% accuracy), with a 1.9 pp accuracy cost, still +43.8 pp above the 41.18% full-coverage baseline. The guardrail is implemented in `remora.selective.guardrail.PhaseAwareGuardrail` and fully tested (25 unit tests).

F.2 PVD-Inspired Offline Agreement Score for Critical-Phase Routing

Kirchner et al. (2024) trained prover-verifier games that produce more legible and reliable LLM outputs: helpful and sneaky *provers* justify claims to iteratively retrained small *verifiers* [2]. REMORA does **not** implement that training game. The component below is a PVD-*inspired*, post-hoc agreement score computed over already-produced oracle responses: it selects two existing responses, scores their agreement with the configured entailment backend (default: token-fingerprint equality, not NLI), and applies a fixed round decay. No model training, live interaction, argument generation, or verifier update occurs, and it is not invoked by the reference API (offline research component).

Protocol.

1. **Cluster** oracle responses via Semantic Entropy clustering [1, 34] (Appendix E), producing semantic equivalence classes $\mathcal{C} = \{C_1, \ldots, C_K\}$ sorted by mass.
2. **Prover** = the oracle whose response belongs to the dominant cluster C_1 and has the highest per-oracle confidence.
3. **Verifier** = the oracle with the highest confidence *outside* C_1 (the independent challenger).
4. **Deliberation** ($r = 1, \ldots, n_{\text{rounds}}$): evaluate bidirectional NLI entailment $e_r = \text{NLI}(\text{prover} \rightarrow \text{verifier}) \cdot \gamma^{r-1}$ with round-decay factor $\gamma = 0.85$.
5. **Legibility score**: $\mathcal{L} = \bar{e} \cdot |C_1|/N$, where $\bar{e}$ is the mean entailment and $|C_1|/N$ is the dominant cluster mass.
6. **Final confidence**: $\hat{c} = \sqrt{|C_1|/N \cdot c_{\text{verifier}}}$ (geometric mean of prover support and updated verifier confidence).

Routing signal. The PVD confidence is blended with the raw REMORA trust score:

$$\tilde{\tau} = (1 - w_{\text{pvd}}) \cdot \tau + w_{\text{pvd}} \cdot \hat{c}, \qquad w_{\text{pvd}} = 0.40 \quad (8)$$

In offline experiments this score can promote borderline critical-phase items to ACCEPT when agreement confidence is high, extending coverage beyond the `PhaseAwareGuardrail` operating point. Neither component is invoked by the reference API, and no formal risk bound applies to this heuristic blend (Appendix G).

Implementation. The deliberation entry point is

```
remora.selective.pvd.deliberate(
    oracle_responses, oracle_confidences,
    n_rounds=2)
```

It returns a `PVDResult` with fields `prover_cluster_mass`, `legibility_score`, `agreement`, and `final_confidence`. `pvd_routing_score(trust_score, pvd_result)` implements Eq. 8. Fully tested (33 unit tests, no external model calls).

G Conformal Risk Control Under Covariate Shift (offline component)

Standard Mondrian conformal calibration assumes that calibration and test samples are *exchangeable* - drawn i.i.d. from the same distribution. This assumption is violated in the critical phase: the joint distribution $(\tau, \text{correct})$ differs structurally between ordered-phase calibration items and critical-phase test items due to the trust-score inversion documented above. Empirically, applying standard conformal to critical-phase items yields 100% observed risk and zero coverage.

REMORA addresses this with a **weighted empirical selective router** *inspired by* Conformal Risk Control (CRC, [3]) but, importantly, *not implementing the CRC procedure* (see below). Each calibration item i receives an importance weight $w_i = p_{\text{test}}(x_i)/p_{\text{cal}}(x_i)$. For REMORA's phase shift, we use the hand-tuned estimate

$$w_i = \begin{cases} 1.0 & \text{if phase}(i) = p_{\text{test}} \\ \beta & \text{otherwise} \end{cases} \qquad (\beta = 0.10) \quad (9)$$

The selector picks the *smallest* threshold $\hat{\lambda}$ (i.e. maximum coverage) such that the weighted empirical risk $\bar{L}(\lambda) = \sum_i \tilde{w}_i \ell_i(\lambda) \leq \alpha$, with $\tilde{w}_i = w_i / \sum_j w_j$ normalised over the calibration items.

Theorem 1 (CRC Guarantee, Angelopoulos et al. 2022, stated for context; not instantiated by the implementation)**.** *For any monotone loss $\ell : [0,1] \to [0,B]$, target risk $\alpha \in (0,B)$, and correctly-specified importance weights:*

$$\mathbb{E}[L(\hat{\lambda})] \leq \alpha + \frac{B}{n+1}$$

where n is the calibration set size.

Non-applicability. The implemented selector does not instantiate this theorem, for two structural reasons no weight specification can repair (external review R3-01): (a) its threshold rule applies the raw constraint $\bar{L}(\lambda) \leq \alpha$ with no $B/(n+1)$ finite-sample term and no test-point weight w_{n+1}; (b) the controlled quantity, the conditional error rate among accepted items, is not monotone in λ, so Theorem 1 does not attach even with valid density-ratio weights. In addition, $\beta = 0.10$ is a hand-tuned constant, not a density ratio [4]. All quality evidence for this component is empirical (repeated-split holdout results).

The component is implemented as `remora.selective.crc.WeightedEmpiricalSelectiveRouter` (formerly `CovariateShiftCRC`, retained as an alias) with a `fit(scores, labels, phases, target_phase)` interface compatible with `ConformalPhaseGuardrail`. A `CRCReport` dataclass exposes `finite_sample_slack` ($= 1/(n_{\text{cal}}+1)$) and `nominal_risk_bound` ($= \alpha +$ slack) as informational reference values (not applicable bounds), plus `holdout_risk` (plain unweighted holdout count). The module is fully tested (45 unit tests) and is an offline research component, not invoked by the reference API.

H Mondrian Conformal Coverage on the Augmented Dataset

Ordered-phase conformal achieves 99.9% coverage with 0/20 failures, a stable internal benchmark operating point (the $N = 2{,}161$ dataset is augmented and includes calibrated-simulation critical items; this is not evidence of deployability under production distribution shift). Critical and disordered phases cannot achieve meaningful coverage via conformal calibration alone, motivating the evidence router.

Table 10: Mondrian phase-stratified conformal, 15% risk target (20 seeds $\times$ $N = 2{,}161$ augmented dataset).

Phase	Mean Risk	Mean Cov.	Seeds Fail
Ordered	12.0%	**99.9%**	**0/20**
Critical	64.5%	0.9%	4/20
Disordered	78.6%	0.2%	2/20

I Optional TEE-Based Audit Hardening (Tamper-Resistant)

This appendix describes tamper-resistant (not tamper-proof) audit hardening via hardware attestation [6]. Implementation status: protocol specified; hardware integration pending. The term "tamper-resistant" is used throughout; the hash-chain in the main system is tamper-detecting only.

I.1 Motivation

The SHA-256 hash-chain in `remora/audit/hash_chain.py` detects post-hoc modification of audit records but does not prevent an adversary with storage write access from replacing the entire chain. For deployments where stronger guarantees are required, hardware attestation via a Trusted Execution Environment (TEE) provides a complementary layer: the `DecisionEnvelope` is signed inside a hardware-isolated enclave by a workload whose measurement matches the approved build and policy digest, so silent substitution is detectable even if storage is compromised. Attestation establishes which code and configuration were measured; it does not establish that the model or the policy decided correctly.

This is an optional deployment hardening; it is not required for the core REMORA governance protocol described in the main paper.

I.2 Supported TEE Platforms

Three platforms are in scope for the reference implementation:

- **Intel TDX / SGX** - remote attestation via DCAP quote generation

- **AMD SEV-SNP** - firmware measurement in attestation report
- **ARM TrustZone** - OP-TEE trusted application (mobile / edge)

I.3 DecisionEnvelope Attestation Format

When TEE hardening is active, each `DecisionEnvelope` includes an `attestation` sub-object:

```
{
  "envelope_id": "env_7f3a...",
  "gate": { "outcome": "ESCALATE", ... },
  "attestation": {
    "tee_platform": "intel_tdx",
    "quote":        "<base64 DCAP quote>",
    "mrenclave":    "<SHA-256 of enclave measurement
    ↪ >",
    "signature":    "<Ed25519 over envelope_id
                     +outcome+timestamp>",
    "timestamp":    "2026-01-15T14:23:07Z"
  }
}
```

The `mrenclave` value pins the exact binary that produced the decision, allowing auditors to verify that no unauthorised code modification occurred between decisions.

I.4 Verification Flow

1. Auditor retrieves envelope and attestation quote from audit log.
2. Quote is verified against Intel/AMD/ARM attestation service.
3. `mrenclave` is compared against the published release measurement.
4. Ed25519 signature over `(envelope_id, outcome, timestamp)` is verified with the enclave's ephemeral key, which is bound to the quote.

If any step fails, the audit record is flagged as potentially tampered.

I.5 Implementation Status

Protocol specified and data contract defined. Hardware integration (enclave compilation, DCAP provisioning, OP-TEE TA signing) is a deployment dependency and is not part of the core source-available release. Stub interfaces are present in `remora/audit/` for downstream integration.